

UNIT 2 - DECISION MAKING AND LOOPING

Python Operators, Operator Precedence, Decision Making: if Statement, if else Statement, if elif else Statement, nested if Statement, Loops: while loop, for loop, nested loops; Loop Control Statements: continue Statement, break Statement, pass Statement: Iterator and Generator

1. OPERATORS AND OPERANDS

- In computer programming languages operators are special symbols which represent computations, conditional matching etc.
- The values the operator uses are called operands.
- Ex: $c = a + b$
- Here a and b are called operands and '+' is an operator
- Python supports following operators.

TYPES OF OPERATORS:

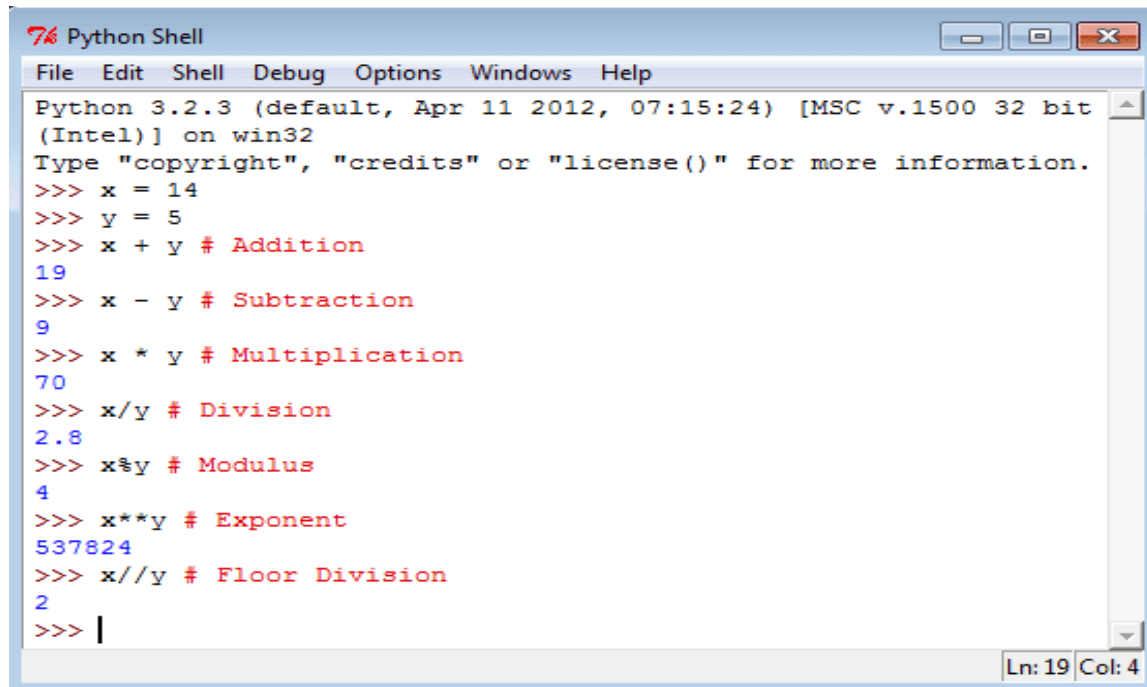
1. Arithmetic Operators
2. Comparison Operators
3. Logical Operators
4. Assignment Operators
5. Bitwise Operator
6. Conditional Operators or ternary operator
7. Membership operators
8. Lambda operator
9. Identity operator

1. Arithmetic Operators

Arithmetic operators are used to perform mathematical operations like addition, subtraction, multiplication, division etc.

Operator	Name	Example	Result
+	Addition	x+y	Sum of x and y.
-	Subtraction	x-y	Difference of x and y.
*	Multiplication	x*y	Product of x and y.
/	Division	x/y	Quotient of x and y.
%	Modulus	x%y	Remainder of x divided by y.
**	Exponent	x**y	x**y will give x to the power y
//	Floor Division	x// y	The division of operands where the result is the quotient in which the digits after the decimal point are removed.

See the following statements in Python Shell.



The screenshot shows a Python Shell window titled "Python Shell" with a menu bar (File, Edit, Shell, Debug, Options, Windows, Help). The shell displays the following code and output:

```
Python 3.2.3 (default, Apr 11 2012, 07:15:24) [MSC v.1500 32 bit
(Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> x = 14
>>> y = 5
>>> x + y # Addition
19
>>> x - y # Subtraction
9
>>> x * y # Multiplication
70
>>> x/y # Division
2.8
>>> x%y # Modulus
4
>>> x**y # Exponent
537824
>>> x//y # Floor Division
2
>>> |
```

The status bar at the bottom right indicates "Ln: 19 Col: 4".

Operation	Result	Example
int/int	Int	$5/2 = 2$
int/float	Float	$5/2.0 = 2.5$
float/int	Float	$5.0/2 = 2.5$
float/float	Float	$5.0/2.0 = 2.5$

Floor Division

- The floor division(//) operator is used to get integer value as result of division operation.
- The floor division(//) operator is meant for integral arithmetic operations as well as floating point arithmetic operations.
- The result of floor division(//) operator can be always floor value of either integer value or float value based on the arguments.
- If both arguments are 'int' type, then the result is 'int' type.
- If atleast one of the argument is float type, then the result is also float type.

NOTE:

- Floor integer value of 3.33333 is 3
- Floor float value of 3.33333 is 3.0
- Floor integer value of 5.9997777 is 5
- Floor float value of 5.9997777 is 5.0

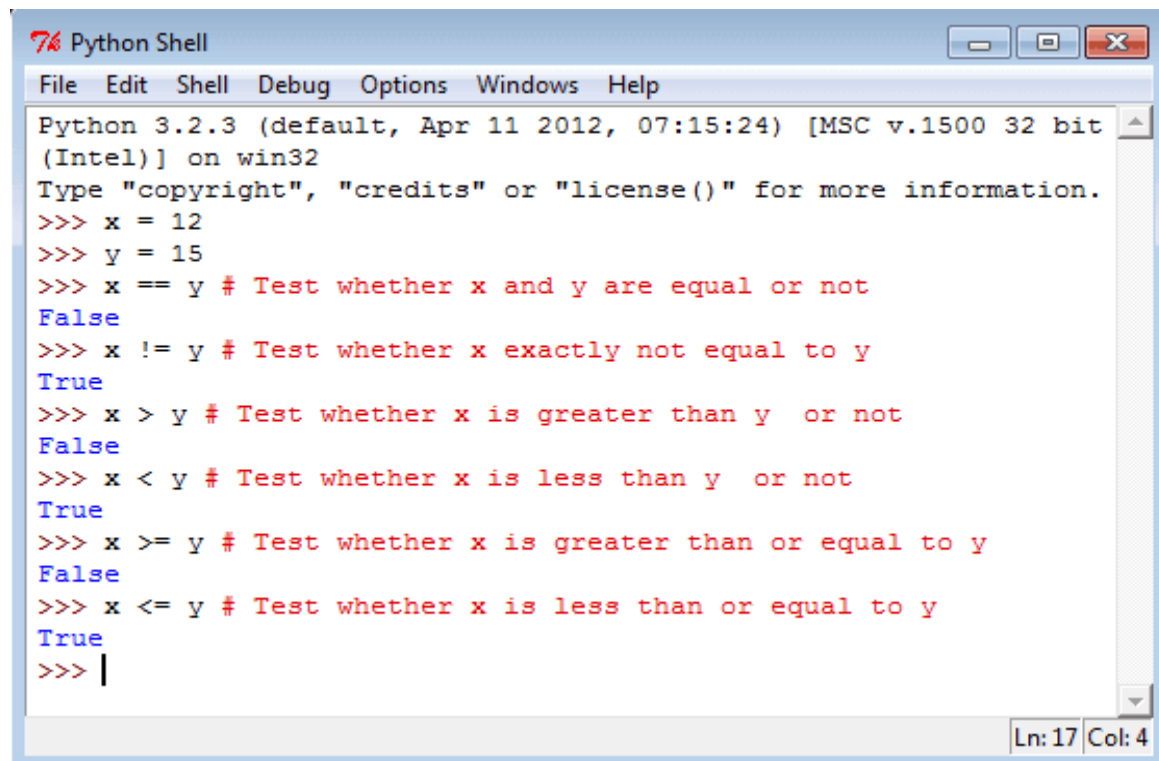
Examples

Command	Output
print(20/2)	10.0
print(20.5/2)	10.25
print(20//2)	10
print(20.5//)	10.0
print(30//2)	15
print(30.0//2)	15.0

2. Relational Operators (or) Comparison Operators

- Comparison operators are used to compare values.
- It either returns True or False according to the condition.

Operator	Name	Example	Result
==	Equal	x==y	True if x is exactly equal to y.
!=	Not equal	x!=y	True if x is exactly not equal to y.
>	Greater than	x>y	True if x (left-hand argument) is greater than y (right-hand argument).
<	Less than	x<y	True if x (left-hand argument) is less than y (right-hand argument).
>=	Greater than or equal to	x>=y	True if x (left-hand argument) is greater than or equal to y (left-hand argument).
<=	Less than or equal to	x<=y	True if x (left-hand argument) is less than or equal to y (right-hand argument).



The screenshot shows a Python Shell window titled "Python Shell" with a menu bar (File, Edit, Shell, Debug, Options, Windows, Help). The shell displays the following text:

```
Python 3.2.3 (default, Apr 11 2012, 07:15:24) [MSC v.1500 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> x = 12
>>> y = 15
>>> x == y # Test whether x and y are equal or not
False
>>> x != y # Test whether x exactly not equal to y
True
>>> x > y # Test whether x is greater than y or not
False
>>> x < y # Test whether x is less than y or not
True
>>> x >= y # Test whether x is greater than or equal to y
False
>>> x <= y # Test whether x is less than or equal to y
True
>>> |
```

The status bar at the bottom right indicates "Ln: 17 Col: 4".

Applying relational operators for 'str' type, comparison is performed based on ASCII Or Unicode values.

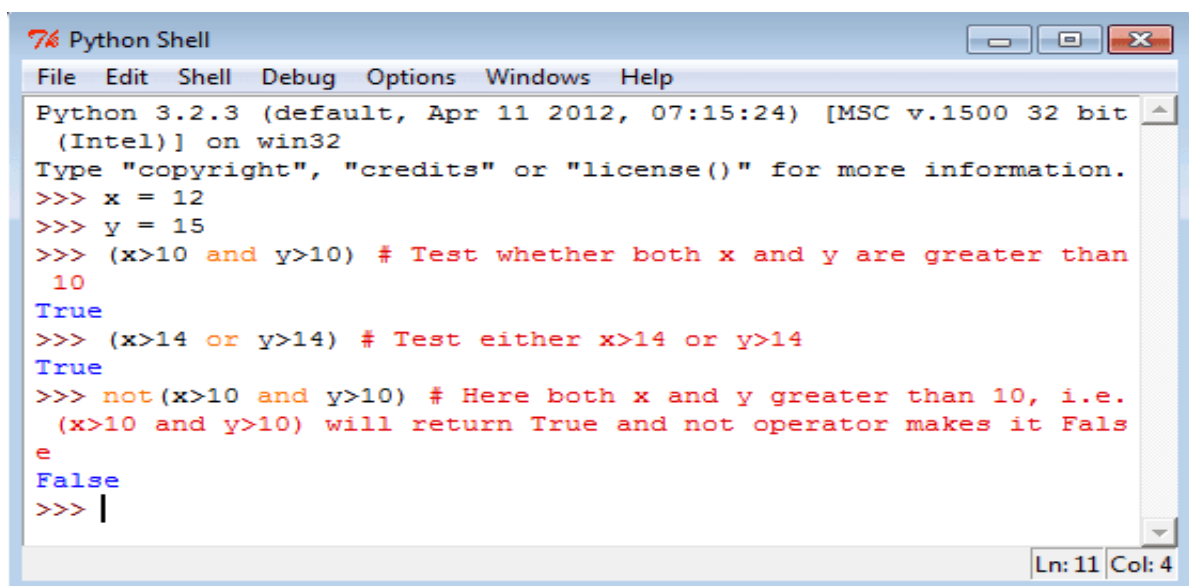
- ✓ To get the ASCII value of any character, use **ord()** function.
- ✓ The ASCII value is known and to find the corresponding character, use the **chr()** function.

Command	Output
<code>print(ord('a'))</code>	97
<code>print(ord('A'))</code>	65
<code>print(chr(97))</code>	a
<code>print(chr(65))</code>	A

3.Logical Operators

Logical operators are typically used with boolean values.

Operator	Example	Result
and	<code>(x and y)</code>	True if both x and y are true.
Or	<code>(x or y)</code>	True if either x or y is true.
Not	<code>(x not y)</code>	If a condition is true then Logical not operator will make false.



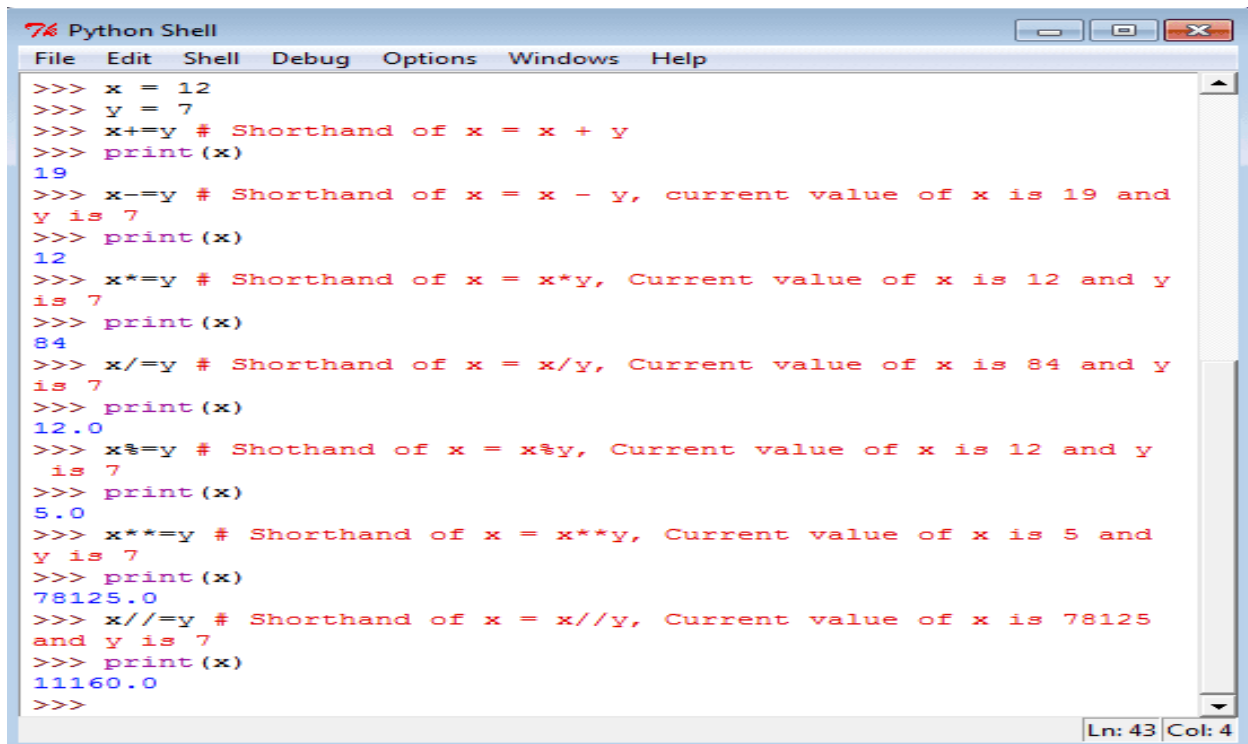
```

Python Shell
File Edit Shell Debug Options Windows Help
Python 3.2.3 (default, Apr 11 2012, 07:15:24) [MSC v.1500 32 bit
(Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> x = 12
>>> y = 15
>>> (x>10 and y>10) # Test whether both x and y are greater than
10
True
>>> (x>14 or y>14) # Test either x>14 or y>14
True
>>> not (x>10 and y>10) # Here both x and y greater than 10, i.e.
(x>10 and y>10) will return True and not operator makes it Fals
e
False
>>> |
  
```

4. Assignment Operators

Assignment operators are used to assign new values to the variables.

Operator	Shorthand	Expression	Description
+=	x+=y	x = x + y	Adds 2 numbers and assigns the result to left operand.
-=	x-= y	x = x -y	Subtracts 2 numbers and assigns the result to left operand.
=	x= y	x = x*y	Multiplies 2 numbers and assigns the result to left operand.
/=	x/= y	x = x/y	Divides 2 numbers and assigns the result to left operand.
%=	x%= y	x = x%y	Computes the modulus of 2 numbers and assigns the result to left operand.
=	x=y	x = x**y	Performs exponential (power) calculation on operators and assign value to the equivalent to left operand.
//=	x//=y	x = x//y	Performs floor division on operators and assign value to the left operand.



```

Python Shell
File Edit Shell Debug Options Windows Help
>>> x = 12
>>> y = 7
>>> x+=y # Shorthand of x = x + y
>>> print(x)
19
>>> x-=y # Shorthand of x = x - y, current value of x is 19 and y is 7
>>> print(x)
12
>>> x*=y # Shorthand of x = x*y, Current value of x is 12 and y is 7
>>> print(x)
84
>>> x/=y # Shorthand of x = x/y, Current value of x is 84 and y is 7
>>> print(x)
12.0
>>> x%=y # Shorthand of x = x%y, Current value of x is 12 and y is 7
>>> print(x)
5.0
>>> x**=y # Shorthand of x = x**y, Current value of x is 5 and y is 7
>>> print(x)
78125.0
>>> x//=y # Shorthand of x = x//y, Current value of x is 78125 and y is 7
>>> print(x)
11160.0
>>>
Ln: 43 Col: 4

```

5. Bitwise Operators

- Bitwise operator works on bits and performs bit by bit operation. These operators are applicable only for int and boolean types. By mistake if we are trying to apply for any other type then we will get Error.

Operator	Shorthand	Expression	Description
&	Bitwise And	$x \& y$	Sets each bit to 1 if both bits are 1
	Bitwise OR	$x y$	Sets each bit to 1 if one of two bits is 1
^	Bitwise XOR	$x \wedge y$	Sets each bit to 1 if only one of two bits is 1
~	Bitwise NOT	$\sim x$	Inverts all the bits
>>	Bitwise Right shift	$x \gg y$	The left operand's value is moved toward right by the number of bits specified by the right operand.
<<	Bitwise Left Shift	$x \ll y$	The left operand's value is moved toward left by the number of bits specified by the right operand.

```
===== RESTART: Shell =====
>>> x = 5
>>> y = 6
>>> z = x and y
>>> print(z)
6
>>> z = x | y
>>> print(z)
7
>>> z = x ^ y
>>> print(z)
3
>>> z = ~x
>>> print(z)
-6
>>> z = ~y
>>> print(z)
-7
>>> z = x << 2
>>> print(z)
20
>>> z = y >> 2
>>> print(z)
1
>>> |
```

6. Conditional Operators or ternary operator

- If the operator operates on only one operand, we will call such operator as **unary** operator. For eg : $\sim a$.
- If the operator operates on Two operands, we will call such operator as **binary** operator. For eg : $a + b$.
- If the operator operates on Three operands, we will call such operator as **Ternary** operator.

Syntax:

x = first Value if condition else second Value

- If condition is True then first Value will be considered else second Value will be considered.

```
a,b=23,43                # a =23  b = 43
c = 50 if a>b else 100
print(c)
```

Output: 100

```
///
>>>
>>> x =5
>>> y =10
>>> # conditional operator
>>> x if (x>y) else y
10
>>>
```

7. Lambda operator or function

- It is just like a macro which means that we can write any operation at lambda function by calling the function with variable name & value would return the final value of operated value.
- Syntax: `lambda_variable_name = lambda variable_name: (operations)`
- `Print(lambda_variable_name (value))`

```

File Edit Shell Debug Options Window Help
Python 3.6.1 (v3.6.1:69c0db5, Mar 21
on win32
Type "copyright", "credits" or "lice:
>>> lfunc=lambda n3:(n3+100)
>>> print(lfunc(10))
110
>>> print(lfunc(101))
201
>>>

```

8. Membership operator

- in and not in are the membership operators in Python. They are used to test whether a value or variable is found in a sequence (string, list, tuple, set and dictionary).
- In a dictionary we can only test for presence of key, not the value.

Operator	Meaning	Example
in	True if value/variable is found in the sequence	5 in x
not in	True if value/variable is not found in the sequence	5 not in x

```

File Edit Format Run Options Window Help
x = "hello world"
y = {1:"a",2:"b"}
print("h in x:", "h" in x)
print("hello not in x:", "hello" not in x)
print("1 in y:", 1 in y)
print("a in y:", "a" in y)

```

Python 3.6

```

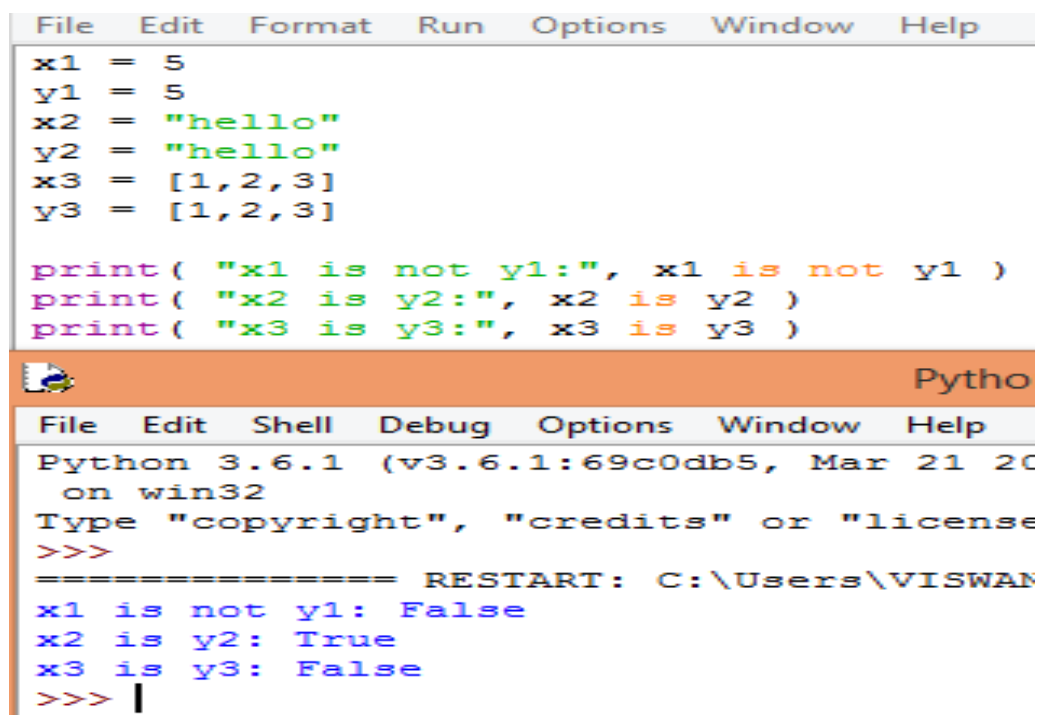
File Edit Shell Debug Options Window Help
Python 3.6.1 (v3.6.1:69c0db5, Mar 21 2017,
on win32
Type "copyright", "credits" or "license()"
>>>
===== RESTART: C:\Users\VISWANATH'
h in x: True
hello not in x: False
1 in y: True
a in y: False
>>>

```

9. Identity operator

- is and is not are the identity operators in Python. They are used to check if two values (or variables) are located on the same part of the memory.

Operator	Meaning	Example
is	True if the operands are identical (refer to the same object)	x is True
is not	True if the operands are not identical (do not refer to the same object)	x is not True



```

File Edit Format Run Options Window Help
x1 = 5
y1 = 5
x2 = "hello"
y2 = "hello"
x3 = [1,2,3]
y3 = [1,2,3]

print( "x1 is not y1:", x1 is not y1 )
print( "x2 is y2:", x2 is y2 )
print( "x3 is y3:", x3 is y3 )

Python 3.6.1 (v3.6.1:69c0db5, Mar 21 2018) on win32
File Edit Shell Debug Options Window Help
Python 3.6.1 (v3.6.1:69c0db5, Mar 21 2018) on win32
Type "copyright", "credits" or "license()" for more
>>>
===== RESTART: C:\Users\VISWANATHAN>>>
x1 is not y1: False
x2 is y2: True
x3 is y3: False
>>> |
  
```

2. ORDER OF OPERATIONS OR PRECEDENCE:

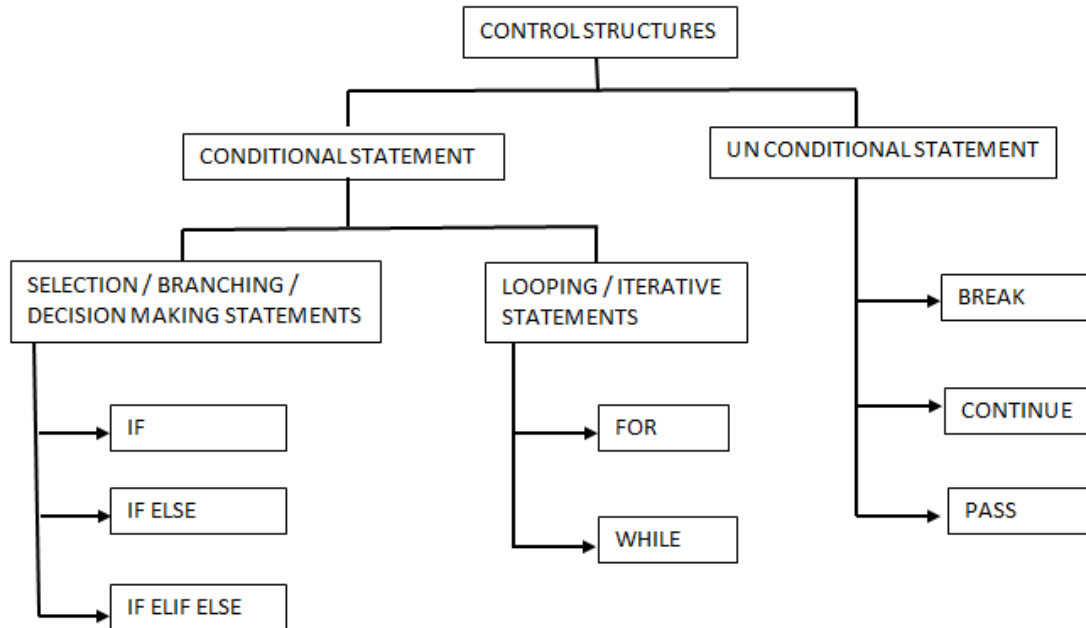
- When an expression contains more than one operator, the order of evaluation depends on the order of operations. For mathematical operators, Python follows mathematical convention. The acronym **PEMDAS** is a useful way to remember the rules:
- **Parentheses** have the highest precedence and can be used to force an expression to evaluate in the order you want. Since expressions in parentheses are evaluated first, $2 * 3 + 4$

(3-1) is 4, and (1+1)**(5-2) is 8. You can also use parentheses to make an expression easier to read, as in (minute * 100) / 60, even if it doesn't change the result.

- **Exponentiation** has the next highest precedence, so $1 + 2^{**}3$ is 9, not 27, and $2 * 3^{**}2$ is 18, not 36.
- **Multiplication and Division** have higher precedence than **Addition and Subtraction**. So $2*3-1$ is 5, not 4, and $6+4/2$ is 8, not 5.
- Operators with the same precedence are evaluated from **left to right** (except exponentiation). So in the expression $\text{degrees} / 2 * \text{pi}$, the division happens first and the result is multiplied by pi. To divide by 2π , you can use parentheses or write $\text{degrees} / 2 / \text{pi}$.

Operator precedence rule in Python	
Operators	Meaning
()	Parentheses
**	Exponent
+x, -x, ~x	Unary plus, Unary minus, Bitwise NOT
*, /, //, %	Multiplication, Division, Floor division, Modulus
+, -	Addition, Subtraction
<<, >>	Bitwise shift operators
&	Bitwise AND
^	Bitwise XOR
	Bitwise OR
==, !=, >, >=, <, <=, is, is not, in, not in	Comparisons, Identity, Membership operators
Not	Logical NOT
And	Logical AND
Or	Logical OR

3. CONTROL STRUCTURES



3.1.3.1 IF STATEMENT / SIMPLE IF / ONE WAY

if statement is the most **simple form** of **decision-making** statement.

It takes an **expression** and **checks**:

- If the expression evaluates to **True** then the block of code in **if statement** will be **executed**.
- If the expression evaluates to **False**, then the **block of code** is **skipped**.

Syntax:

if expression :

statement_1

statement_2

....

In the case of if expression specifies the conditions which are based on Boolean expression. When a Boolean expression is evaluated it produces either a value of true or false. If the expression evaluates true the same amount of indented statement(s) of if will be executed. This group of the statement(s) is called a block.

FLOW CHART

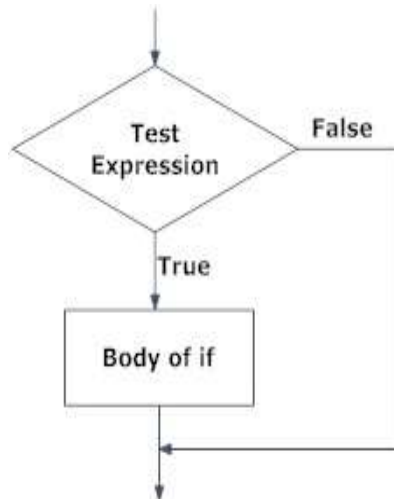
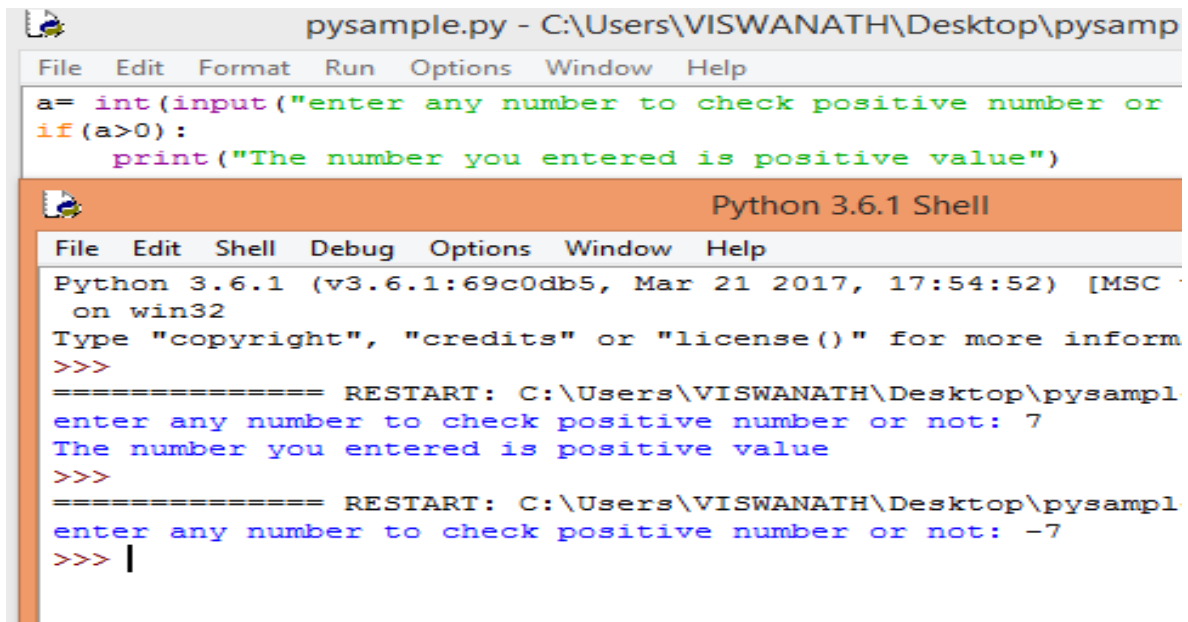


Fig: Operation of if statement

Examples:

Program to provide bonus mark if the category is Sports	output
<pre> m=eval(input("enter ur mark out of 100")) c=input("enter ur category G/S") if(c=="S"): m=m+5 print("mark is",m) </pre>	<pre> enter ur mark out of 100 85 enter ur category G/S S mark is 90 </pre>

Example: Check whether the entered number is +ve or not



The image shows two windows. The top window is a text editor titled 'pysample.py - C:\Users\VISWANATH\Desktop\pysamp' containing the following Python code:

```
a= int(input("enter any number to check positive number or
if(a>0):
    print("The number you entered is positive value")
```

The bottom window is a 'Python 3.6.1 Shell' showing the execution of the script. It displays the prompt 'enter any number to check positive number or not:' followed by the input '7', and then the output 'The number you entered is positive value'. It also shows a second execution with input '-7'.

3.1.3.2 IF ELSE STATEMENT/ALTERNATIVE WAY/TWO WAY

In the alternative the condition must be true or false. In this, **else** statement is combined with **if** statement.

- If the condition is true, statements inside the **if** get executed otherwise else part gets executed.
- The **else** statement contains the block of code that executes when the condition is false. The alternatives are called branches, because they are branches in the flow of execution.

Syntax:

if expression :

statement_1

statement_2

....

else:

statement_3

statement_4

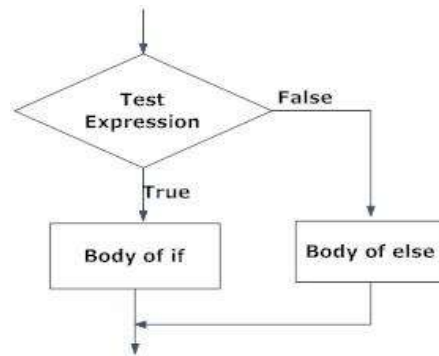
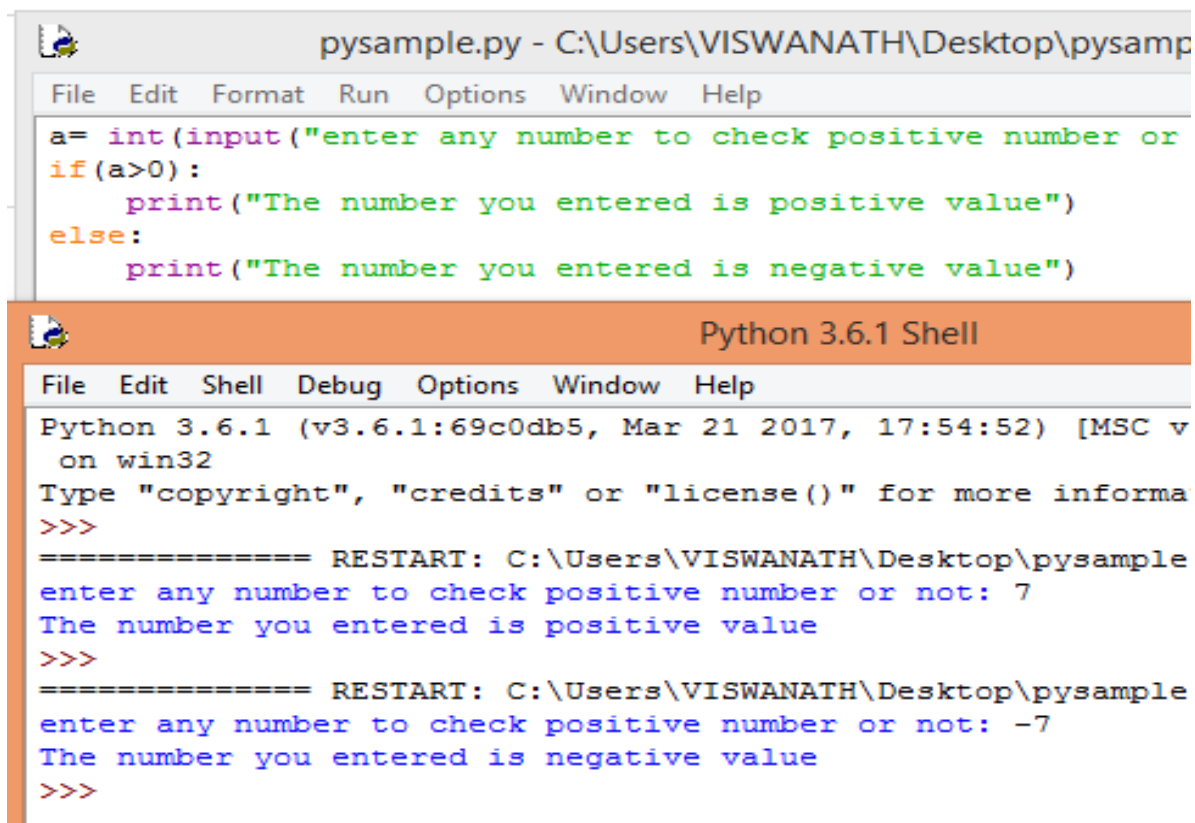
FLOWCHART:

Fig: Operation of if...else statement

Examples:

Odd or even number	Output
<pre>n=eval(input("enter a number")) if(n%2==0): print("even number") else: print("odd number")</pre>	<pre>enter a number 4 even number</pre>
positive or negative number	Output
<pre>n=eval(input("enter a number")) if(n>=0): print("positive number") else: print("negative number")</pre>	<pre>enter a number 8 positive number</pre>
leap year or not	Output
<pre>y=eval(input("enter a year")) if(y%4==0): print("leap year") else: print("not leap year")</pre>	<pre>enter a year 2000 leap year</pre>



The screenshot displays two windows from a Python IDE. The top window, titled 'pysample.py - C:\Users\VISWANATH\Desktop\pysamp', contains the following Python code:

```
a= int(input("enter any number to check positive number or  
if(a>0):  
    print("The number you entered is positive value")  
else:  
    print("The number you entered is negative value")
```

The bottom window, titled 'Python 3.6.1 Shell', shows the execution of the script. It includes the Python version and build information, followed by two restarts of the script. In the first run, the user enters '7' and the output is 'The number you entered is positive value'. In the second run, the user enters '-7' and the output is 'The number you entered is negative value'.

```
Python 3.6.1 (v3.6.1:69c0db5, Mar 21 2017, 17:54:52) [MSC v  
on win32  
Type "copyright", "credits" or "license()" for more informa  
>>>  
===== RESTART: C:\Users\VISWANATH\Desktop\pysample  
enter any number to check positive number or not: 7  
The number you entered is positive value  
>>>  
===== RESTART: C:\Users\VISWANATH\Desktop\pysample  
enter any number to check positive number or not: -7  
The number you entered is negative value  
>>>
```

3.1.3.3 IF ELIF STATEMENT / CHAINED / MULTIPLE ALTERNATIVES

- The elif is short for else if.
- This is used to check more than one condition.
- If the condition1 is False, it checks the condition2 of the elif block. If all the conditions are False, then the else part is executed.
- Among the several if...elif...else part, only one part is executed according to the condition.

The if block can have only one else block. But it can have multiple elif blocks. The way to express a computation like that is a chained conditional.

Syntax:

if expression1 :

 statement_1

 statement_2

elif expression2 :

 statement_3

 statement_4

elif expression3 :

 statement_5

 statement_6

else:

 statement_n

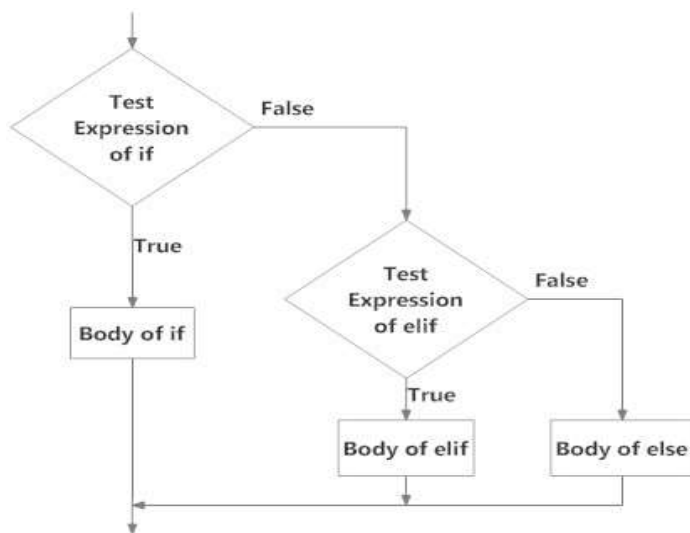
Flowchart:

Fig: Operation of if...elif...else statement

In the above case, Python evaluates each expression (i.e. the condition) one by one and if a true condition is found, the statement(s) block under that expression will be executed. If no true condition is found, the statement(s) block under else will be executed. In the following example, we have applied if, series of elif and else to get the grade.

Examples:

```
if score >= 90.0:
    grade = 'A'
else:
    if score >= 80.0:
        grade = 'B'
    else:
        if score >= 70.0:
            grade = 'C'
        else:
            if score >= 60.0:
                grade = 'D'
            else:
                grade = 'F'
```

(a)

Equivalent

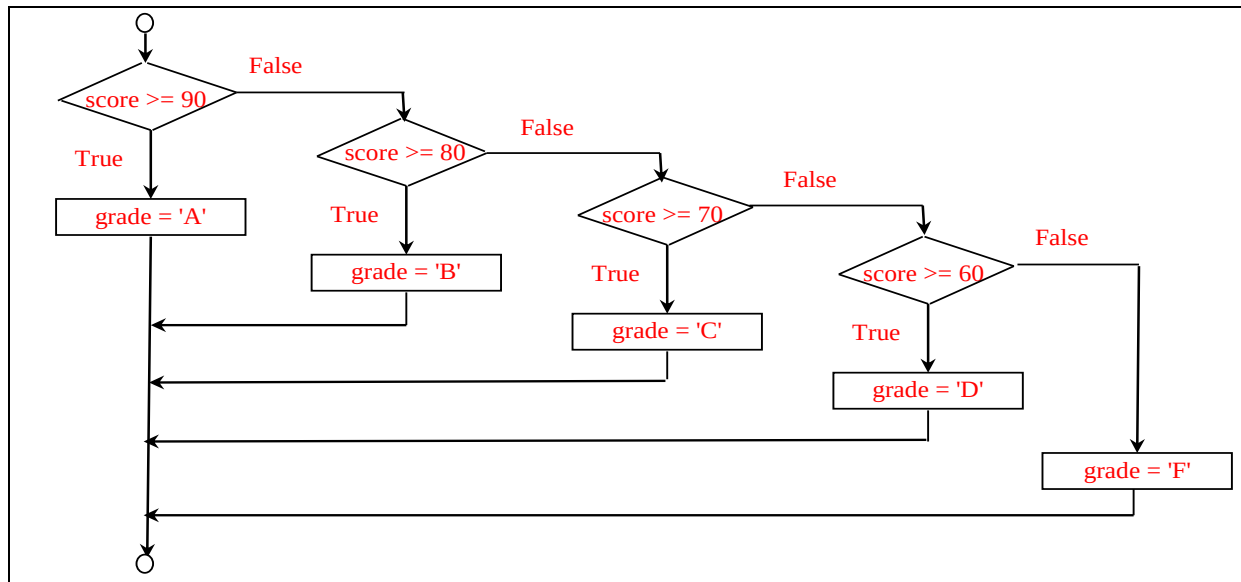
This is better

```
if score >= 90.0:
    grade = 'A'
elif score >= 80.0:
    grade = 'B'
elif score >= 70.0:
    grade = 'C'
elif score >= 60.0:
    grade = 'D'
else:
    grade = 'F'
```

(b)

```
File Edit Format Run Options Window Help
mark1,mark2,mark3 = eval(input("ENTER MARK1, MARK2,MARK3 seperated by comma: "))
total = mark1 + mark2 + mark3
average = total / 3
print("toatal=",total)
print("Average = ", average)
if average >= 90.0:
    print("grade:A")
elif average >= 80.0:
    print("grade:B")
elif average >= 70.0:
    print("grade:c")
elif average >= 60.0:
    print("grade:D")
else:
    print("grade:FAIL")
```

```
Python 3.6.1 Shell
File Edit Shell Debug Options Window Help
Python 3.6.1 (v3.6.1:69c0db5, Mar 21 2017, 17:54:52) [MSC v.1900 32 bit (Intel)]
on win32
Type "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:\Users\VISWANATH\Desktop\pysample.py =====
ENTER MARK1, MARK2,MARK3 seperated by comma: 50,60,70
toatal= 180
Average = 60.0
grade:D
>>>
```



Use of and operator in if statement:

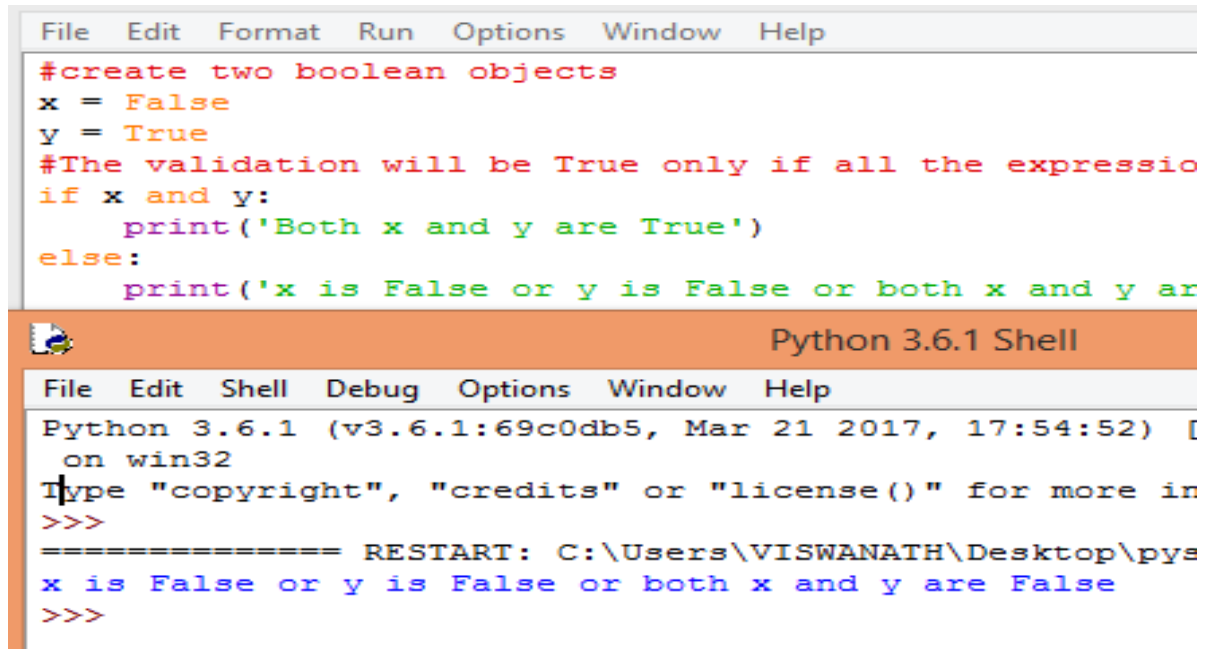
Suppose if condition has to satisfy two or more condition then we have to use and operator to check the condition of both the expressions. If you use and operator in if condition, then both the conditions must be true for the execution of if block. If any one condition fails, then 'if' block is not executed.

Truth table for AND:

p1	p2	p1 and p2	Example (assume age = 24, gender = 'F')
False	False	False	(age > 18) and (gender == 'F') is True, because (age > 18) and (gender == 'F') are both True.
False	True	False	
True	False	False	(age > 18) and (gender != 'F') is False, because (gender != 'F') is False.
True	True	True	

Truth table for OR

p1	p2	p1 or p2	Example (assume age = 24, gender = 'F')
False	False	False	(age > 34) or (gender == 'F') is true, because (gender == 'F') is True.
False	True	True	
True	False	True	(age > 34) or (gender == 'M') is False, because (age > 34) and (gender == 'M') are both False.
True	True	True	

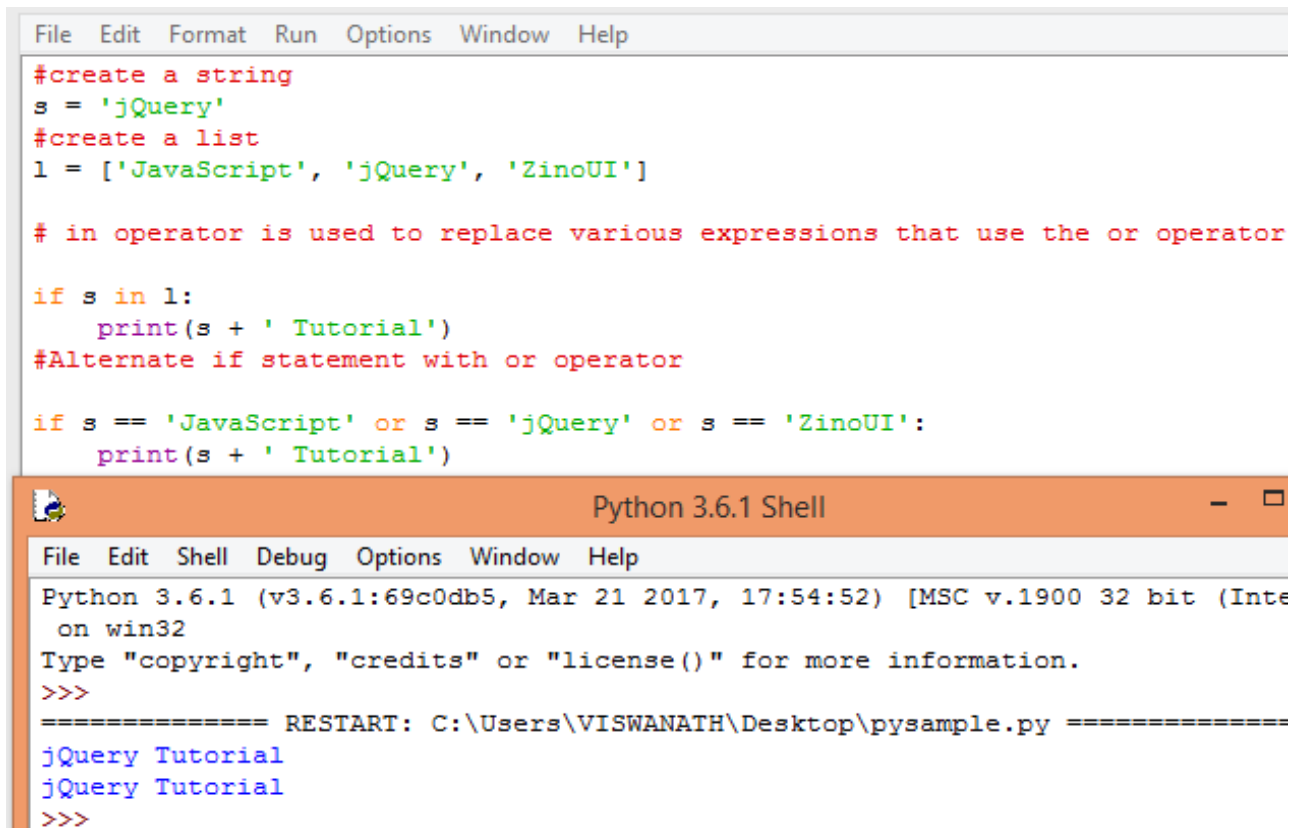


```
File Edit Format Run Options Window Help
#create two boolean objects
x = False
y = True
#The validation will be True only if all the expressions are True
if x and y:
    print('Both x and y are True')
else:
    print('x is False or y is False or both x and y are False')
```

Python 3.6.1 Shell

```
File Edit Shell Debug Options Window Help
Python 3.6.1 (v3.6.1:69c0db5, Mar 21 2017, 17:54:52) [AMD64] on win32
Type "copyright", "credits" or "license()" for more in
>>>
===== RESTART: C:\Users\VISWANATH\Desktop\pysample.py =====
x is False or y is False or both x and y are False
>>>
```

Use of in operator in an if statement



```
File Edit Format Run Options Window Help
#create a string
s = 'jQuery'
#create a list
l = ['JavaScript', 'jQuery', 'ZinoUI']

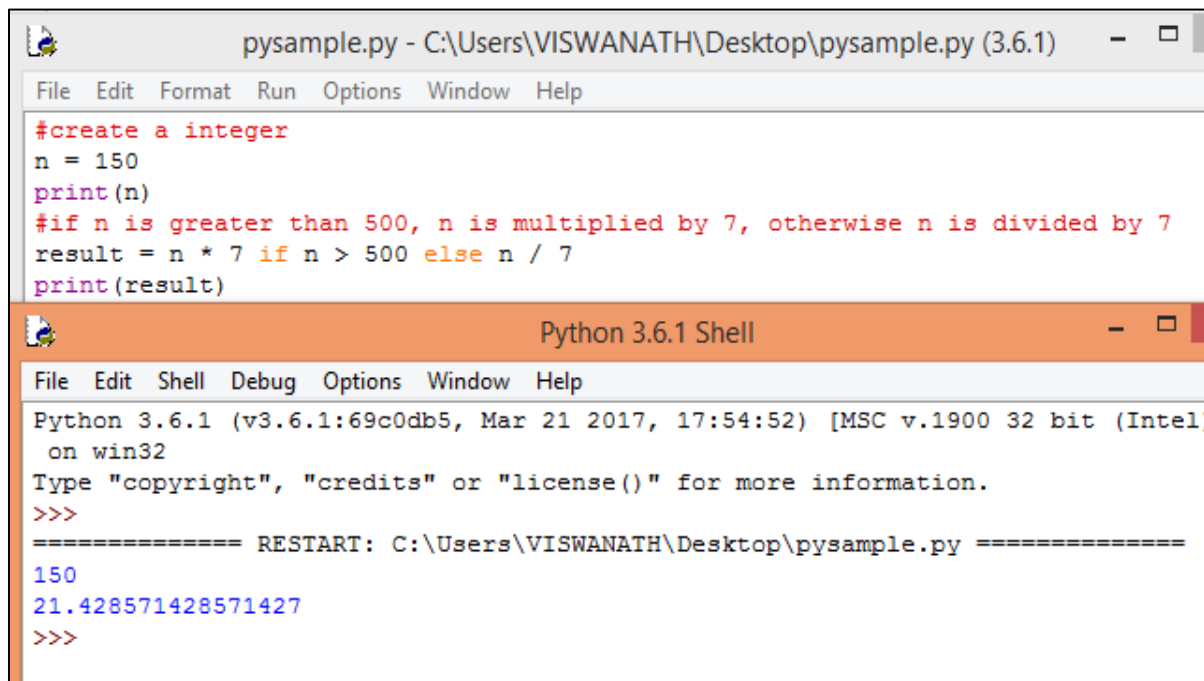
# in operator is used to replace various expressions that use the or operator

if s in l:
    print(s + ' Tutorial')
#Alternate if statement with or operator

if s == 'JavaScript' or s == 'jQuery' or s == 'ZinoUI':
    print(s + ' Tutorial')
```

Python 3.6.1 Shell

```
File Edit Shell Debug Options Window Help
Python 3.6.1 (v3.6.1:69c0db5, Mar 21 2017, 17:54:52) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:\Users\VISWANATH\Desktop\pysample.py =====
jQuery Tutorial
jQuery Tutorial
>>>
```

if-else in a single line of code

The screenshot shows a Python IDE window titled 'pysample.py - C:\Users\VISWANATH\Desktop\pysample.py (3.6.1)'. The code in the editor is as follows:

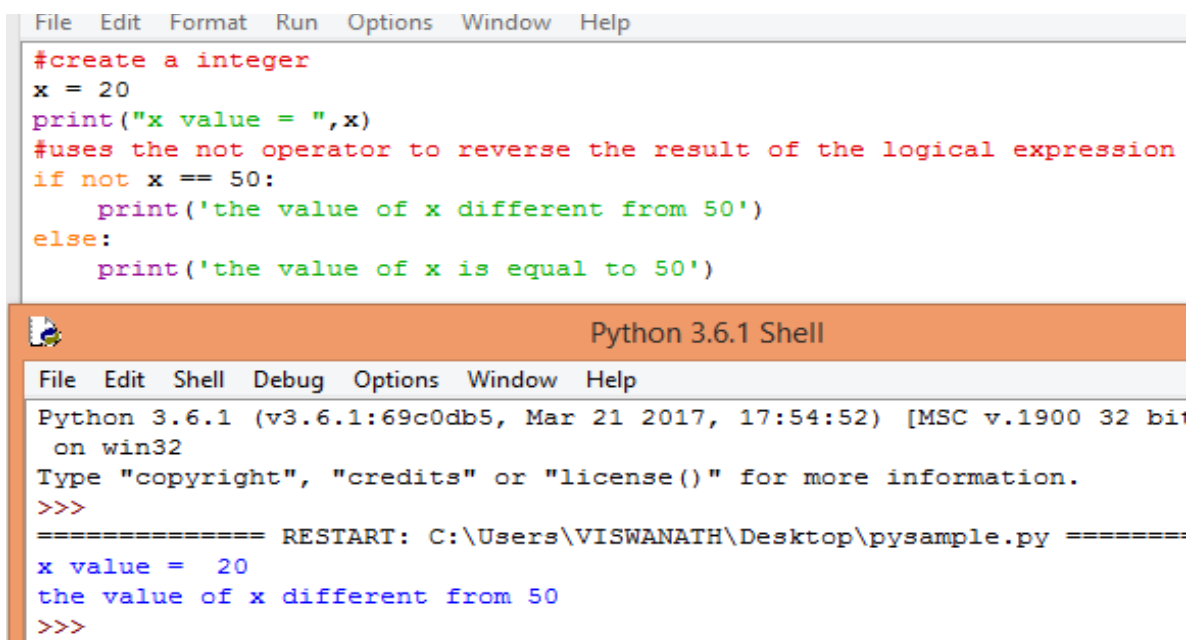
```
#create a integer
n = 150
print(n)
#if n is greater than 500, n is multiplied by 7, otherwise n is divided by 7
result = n * 7 if n > 500 else n / 7
print(result)
```

Below the editor is the 'Python 3.6.1 Shell' window. It shows the execution output:

```
Python 3.6.1 (v3.6.1:69c0db5, Mar 21 2017, 17:54:52) [MSC v.1900 32 bit (Intel) on win32]
Type "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:\Users\VISWANATH\Desktop\pysample.py =====
150
21.428571428571427
>>>
```

Define a negative if

If a condition is true, the not operator is used to reverse the logical state. The logical not operator will make it false.



The screenshot shows a Python IDE window titled 'Python 3.6.1 Shell'. The code in the editor is as follows:

```
#create a integer
x = 20
print("x value = ",x)
#uses the not operator to reverse the result of the logical expression
if not x == 50:
    print('the value of x different from 50')
else:
    print('the value of x is equal to 50')
```

Below the editor is the 'Python 3.6.1 Shell' window. It shows the execution output:

```
Python 3.6.1 (v3.6.1:69c0db5, Mar 21 2017, 17:54:52) [MSC v.1900 32 bit (Intel) on win32]
Type "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:\Users\VISWANATH\Desktop\pysample.py =====
x value = 20
the value of x different from 50
>>>
```

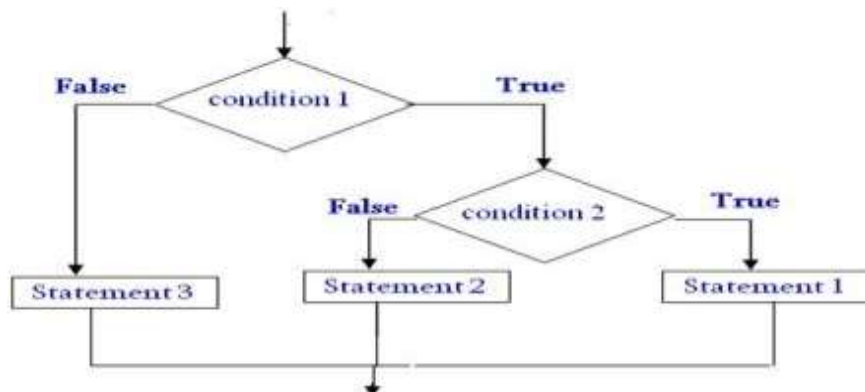
Nested conditionals

One conditional can also be nested within another. Any number of condition can be nested inside one another. In this, if the condition is true it checks another if condition1. If both the conditions are true statement1 get executed otherwise statement2 get execute. if the condition is false statement3 gets executed.

Syntax

```
if (condition):  
    if(condition 1):  
        statement 1  
    else:  
        statement 2  
else:  
    statement 3
```

Flowchart



Example:

Greatest of three numbers

```
a=eval(input("enter the value of a"))
```

```
b=eval(input("enter the value of b"))
```

```
c=eval(input("enter the value of c"))
```

```
if(a>b):
```

```
    if(a>c):
```

```
        print("the greatest no is",a)
```

```
    else:
```

```
        print("the greatest no is",c)
else:
    if(b>c):
        print("the greatest no is",b)
    else:
        print("the greatest no is",c)
```

output

enter the value of a 9
enter the value of a 1
enter the value of a 8
the greatest no is 9

3.2 LOOPING**Definition of Loop / Iteration:**

Looping is defined as repeatedly executing block of statements for certain number of times.

Python has mainly 2 types of looping statements.

- For loop
- While loop

3.2.1. FOR LOOP

Like most other languages, Python has for loops, but it differs a bit from C or Pascal. It is entry controlled loop.

- In Python , for loop is used to iterate over the items of any sequence including the Python list, string, tuple etc.
- The for loop is also used to access elements from a container (for example list, string, tuple) using built-in function range().

Syntax: General form of for loop:

for variable_name in sequence :

statement_1

statement_2

Parameter

Name	Description
variable_name	It indicates target variable which will set a new value for each iteration of the loop.
sequence	A sequence of values that will be assigned to the target variable variable_name. Values are provided using a list or a string or from the built-in function range().
statement_1, statement_2	Block of program statements.

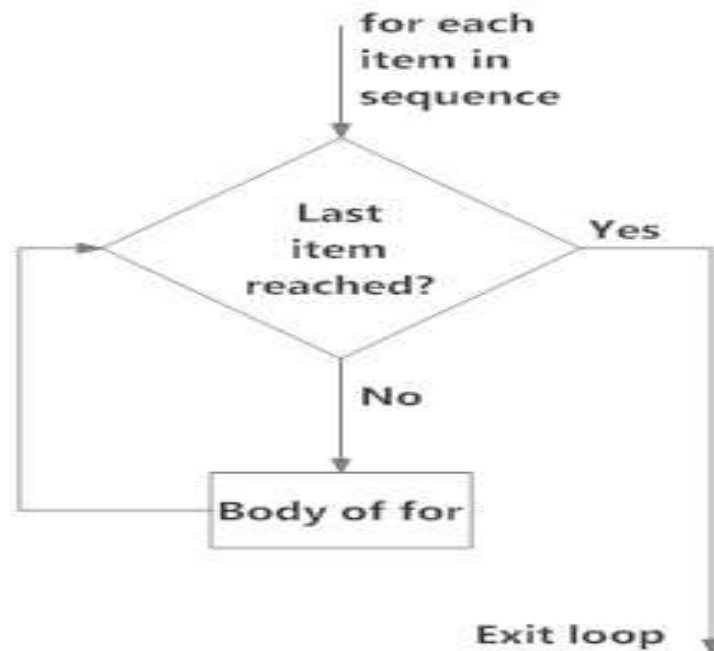
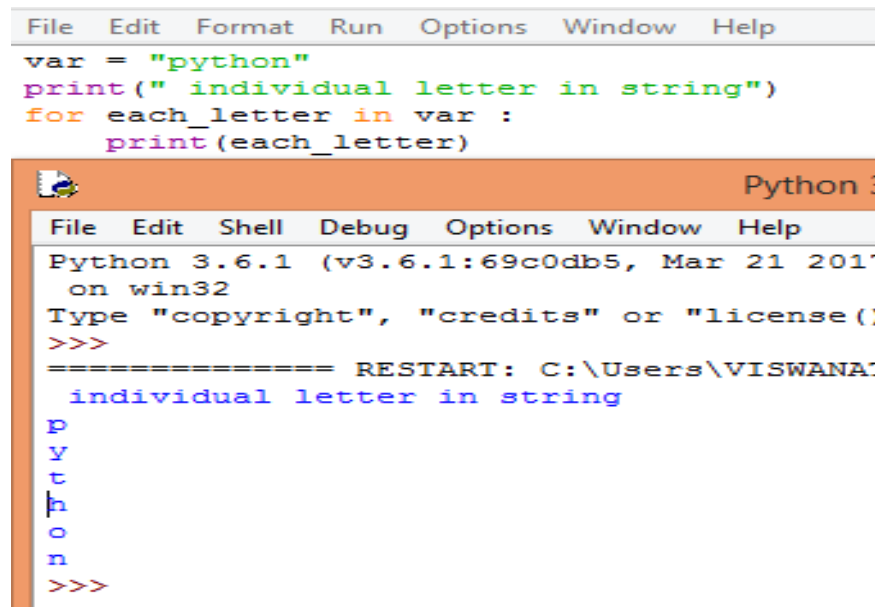
Flowchart:

Fig: operation of for loop

Example:

```
File Edit Format Run Options Window Help
var = "python"
print(" individual letter in string")
for each_letter in var :
    print(each_letter)
```

```
Python 3.6.1 (v3.6.1:69c0db5, Mar 21 2017) on win32
Type "copyright", "credits" or "license()"
>>>
===== RESTART: C:\Users\VISWANATHAN>
individual letter in string
p
y
t
h
o
n
>>>
```

Syntax:**Method 2: for loop with range function**

```
for variable_name in range(initial, final, step value):
    statement_1
    statement_2
```

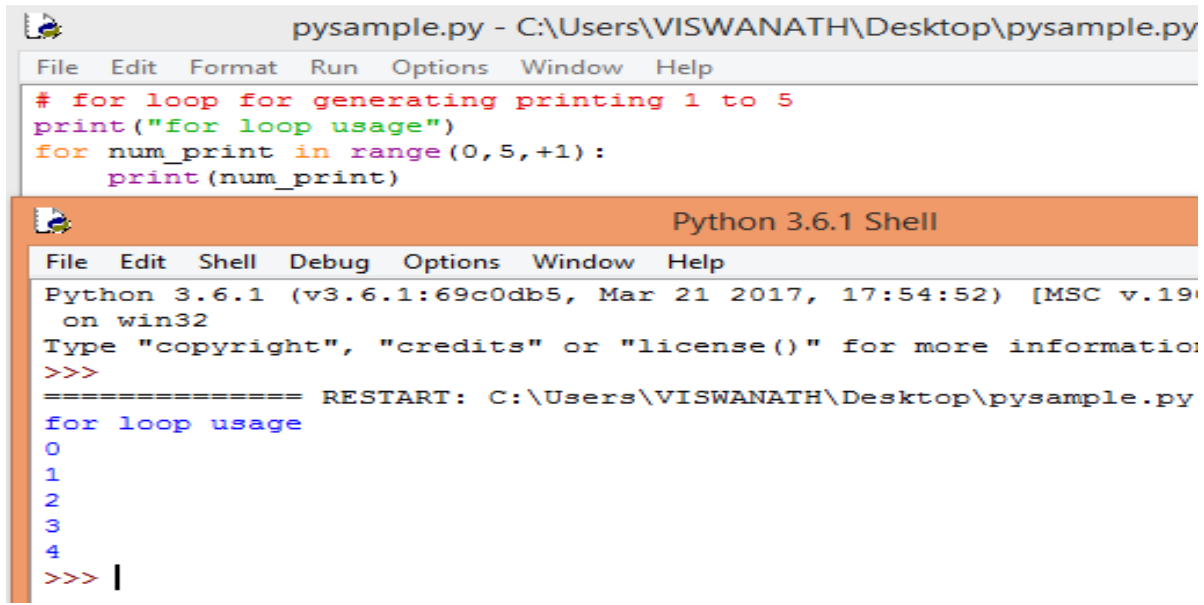
Conditions for range functions:

Initial / begin value: Set the starting value for the loop. It is optional i.e., value may be omitted. The default start value is 0.

Final / End value: Set the last value for the loop. End value is mandatory to declare. The value which you are giving as end is considered as end_value = end_value -1

Increment / Decrement step value: Indicate the number of steps needed to increment or decrement value. Default step value is +1.

The above mentioned 3 values accept only integer values not floating point values.



The screenshot displays two windows from a Python IDE. The top window, titled 'pysample.py - C:\Users\VISWANATH\Desktop\pysample.py', contains the following code:

```
# for loop for generating printing 1 to 5
print("for loop usage")
for num_print in range(0,5,+1):
    print(num_print)
```

The bottom window, titled 'Python 3.6.1 Shell', shows the execution output:

```
Python 3.6.1 (v3.6.1:69c0db5, Mar 21 2017, 17:54:52) [MSC v.191
on win32
Type "copyright", "credits" or "license()" for more information
>>>
===== RESTART: C:\Users\VISWANATH\Desktop\pysample.py
for loop usage
0
1
2
3
4
>>> |
```

Python for loop and range() function

The range() function returns a list of consecutive integers. The function has one, two or three parameters where last two parameters are optional. It is widely used in for loops.

Syntax:

```
range(10) --> 0,1,2,3,4,5,6,7,8,9
range(1, 10) --> 1,2,3,4,5,6,7,8,9
range(1, 10, 2) --> 1,3,5,7,9
range(10, 0, -1) --> 10,9,8,7,6,5,4,3,2,1
range(10, 0, -2) --> 10,8,6,4,2
range(2, 11, 2) --> 2,4,6,8,10
range(-5, 5) --> -5,-4,-3,-2,-1,0,1,2,3,4
range(1, 2) --> 1
range(1, 1) --> (empty)
range(1, -1) --> (empty)
range(1, -1, -1) --> 1,0
range(0) --> (empty)
```

```

print("number generated based on various range function input is")

print("range(10):")
for num in range(10):
    print(num,end=",")
print("\n")
print("range(1, 10) is:")
for num in range(1, 10):
    print(num,end=",")
print("\n")
print("range(1, 10, 2) is:")
for num in range(1, 10, 2):
    print(num,end=",")
print("\n")
print("range(10, 0, -1) is:")
for num in range(10, 0, -1):
    print(num,end=",")
print("\n")
print("range(10, 0, -2) is:")
for num in range(10, 0, -2):
    print(num,end=",")
print("\n")
print("range(2, 11, 2) is:")
for num in range(2, 11, 2):
    print(num,end=",")
print("\n")
print("range(-5, 5) is:")
for num in range(-5, 5):
    print(num,end=",")
print("\n")
print("range(1, 2) is:")
for num in range(1, 2):
    print(num,end=",")
print("\n")
print("range(1, 1) is:")
for num in range(1, 1):
    print(num,end=",")
print("\n")
print("range(1, -1) is:")
for num in range(1, -1):
    print(num,end=",")

```

Python 3.6

File Edit Shell Debug Options Window Help

```

>>>
===== RESTART: C:\Users\VISWANATH\
number generated based on various range fun
range(10):
0,1,2,3,4,5,6,7,8,9,

range(1, 10) is:
1,2,3,4,5,6,7,8,9,

range(1, 10, 2) is:
1,3,5,7,9,

range(10, 0, -1) is:
10,9,8,7,6,5,4,3,2,1,

range(10, 0, -2) is:
10,8,6,4,2,

range(2, 11, 2) is:
2,4,6,8,10,

range(-5, 5) is:
-5,-4,-3,-2,-1,0,1,2,3,4,

range(1, 2) is:
1,

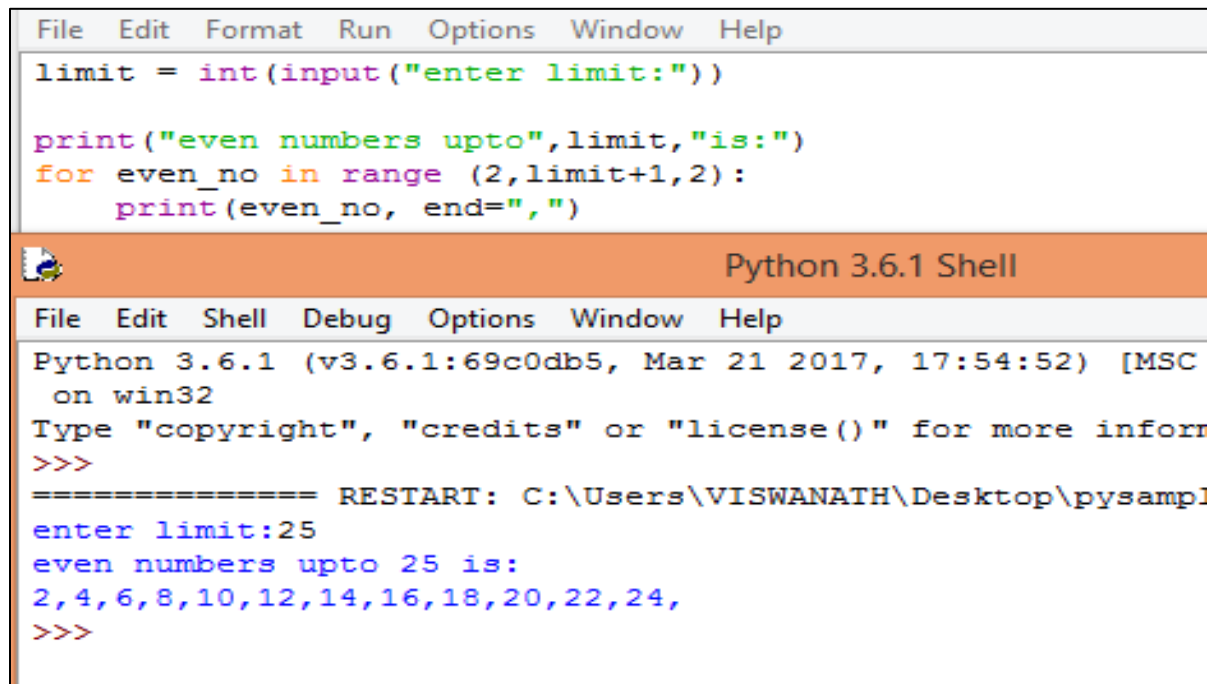
range(1, 1) is:

range(1, -1) is:

range(1, -1, -1) is:
1,0,

range(0) is:

```

Example: Program to print even numbers up to n using for loop

The screenshot shows a Python IDE with a menu bar (File, Edit, Format, Run, Options, Window, Help) and a code editor. The code in the editor is as follows:

```
limit = int(input("enter limit:"))

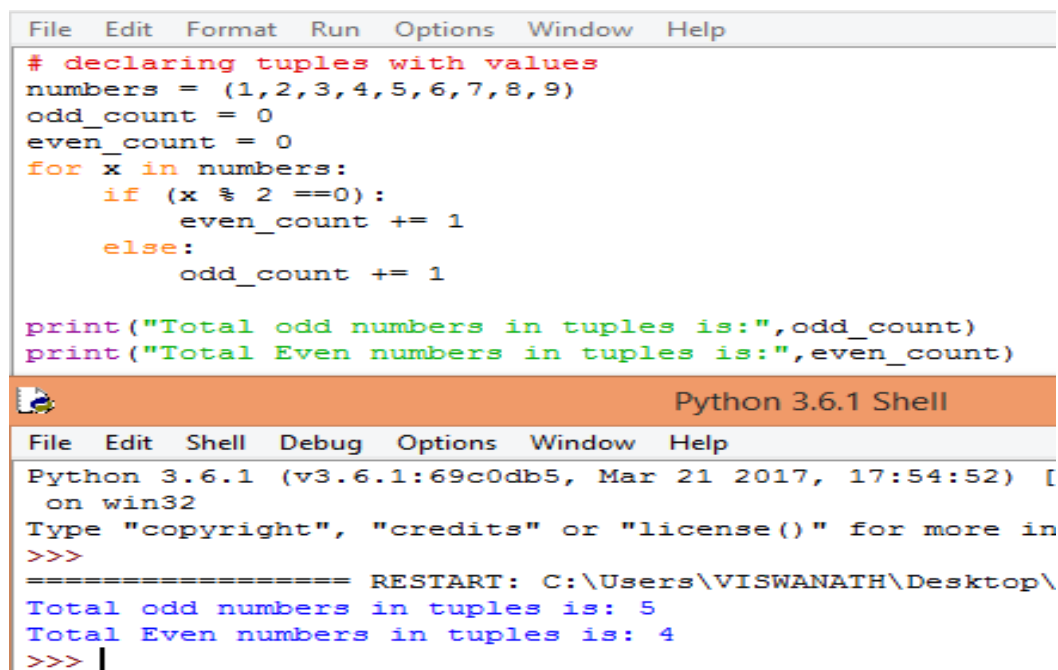
print("even numbers upto",limit,"is:")
for even_no in range (2,limit+1,2):
    print(even_no, end=",")
```

Below the code editor is a terminal window titled "Python 3.6.1 Shell" with a menu bar (File, Edit, Shell, Debug, Options, Window, Help). The terminal output is as follows:

```
Python 3.6.1 (v3.6.1:69c0db5, Mar 21 2017, 17:54:52) [MSC
on win32
Type "copyright", "credits" or "license()" for more inform
>>>
===== RESTART: C:\Users\VISWANATH\Desktop\pysamp1
enter limit:25
even numbers upto 25 is:
2,4,6,8,10,12,14,16,18,20,22,24,
>>>
```

Python for loop: Iterating over tuple, list, dictionary**Example: Iterating over tuple**

The following example counts the number of even and odd numbers from a series of numbers.



The screenshot shows a Python IDE with a menu bar (File, Edit, Format, Run, Options, Window, Help) and a code editor. The code in the editor is as follows:

```
# declaring tuples with values
numbers = (1,2,3,4,5,6,7,8,9)
odd_count = 0
even_count = 0
for x in numbers:
    if (x % 2 ==0):
        even_count += 1
    else:
        odd_count += 1

print("Total odd numbers in tuples is:",odd_count)
print("Total Even numbers in tuples is:",even_count)
```

Below the code editor is a terminal window titled "Python 3.6.1 Shell" with a menu bar (File, Edit, Shell, Debug, Options, Window, Help). The terminal output is as follows:

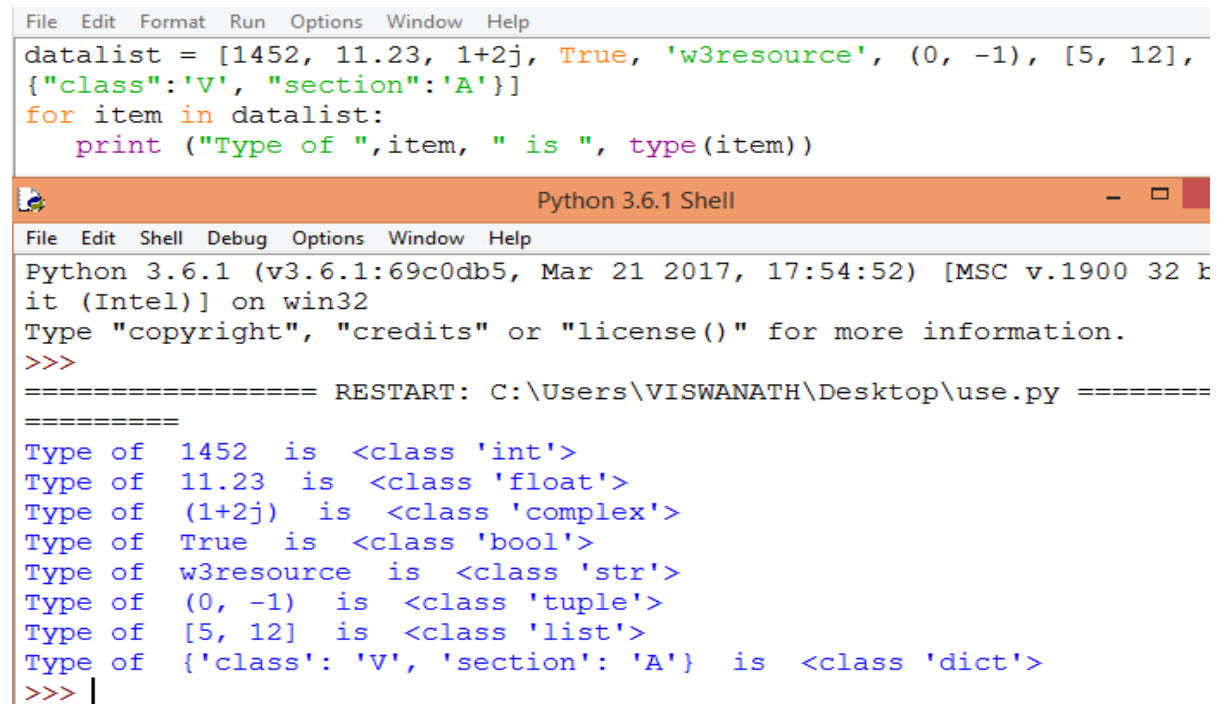
```
Python 3.6.1 (v3.6.1:69c0db5, Mar 21 2017, 17:54:52) [
on win32
Type "copyright", "credits" or "license()" for more in
>>>
===== RESTART: C:\Users\VISWANATH\Desktop\
Total odd numbers in tuples is: 5
Total Even numbers in tuples is: 4
>>> |
```

In the above example a tuple named numbers is declared which holds the integers 1 to 9.

- The best way to check if a given number is even or odd is to use the modulus operator (%). The % operator returns the remainder when dividing two numbers. Modulus of $8 \% 2$ returns 0 as 8 is divided by 2, therefore 8 is even and modulus of $5 \% 2$ returns 1 therefore 5 is odd.
- The for loop iterates through the tuple and we test modulus of $x \% 2$ is true or not, for every item in the tuple and the process will continue until we reach the end of the tuple. When it is true count_even is increased by one otherwise count_odd is increased by one. Finally, we print the number of even and odd numbers through print statements.

Example: Iterating over list

In the following example for loop iterates through the list "datalist" and prints each item and its corresponding Python type.



```
File Edit Format Run Options Window Help
datalist = [1452, 11.23, 1+2j, True, 'w3resource', (0, -1), [5, 12],
{"class": 'V', "section": 'A'}]
for item in datalist:
    print ("Type of ",item, " is ", type(item))

Python 3.6.1 Shell
File Edit Shell Debug Options Window Help
Python 3.6.1 (v3.6.1:69c0db5, Mar 21 2017, 17:54:52) [MSC v.1900 32 b
it (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:\Users\VISWANATH\Desktop\use.py =====
=====
Type of 1452 is <class 'int'>
Type of 11.23 is <class 'float'>
Type of (1+2j) is <class 'complex'>
Type of True is <class 'bool'>
Type of w3resource is <class 'str'>
Type of (0, -1) is <class 'tuple'>
Type of [5, 12] is <class 'list'>
Type of {'class': 'V', 'section': 'A'} is <class 'dict'>
>>> |
```

Example: Iterating over dictionary

In the following example for loop iterates through the dictionary "color" through its keys and prints each key.

```
color = {"c1": "Red", "c2": "Green", "c3": "Orange"}
for key in color:
    print(key)
```

Output:

```
c3
c2
c1
```

Following for loop iterates through its values:

```
color = {"c1": "Red", "c2": "Green", "c3": "Orange"}
for value in color.values():
    print(value)
```

Output:

```
Orange
Green
Red
```

You can attach an optional else clause with for statement.

In this case,

syntax will be:

For variable_name in sequence:

```
statement_1
statement_2
....
```

else:

```
statement_3
statement_4
....
```

The else clause is only executed after completing the for loop. If a break statement executes in first program block and terminates the loop then the else clause will not execute.

3.1.3.5 WHILE LOOP

The basic loop structure in Python is while loop. It is entry controlled loop.

Syntax :

while (expression) :

statement_1

statement_2

....

Flow chart

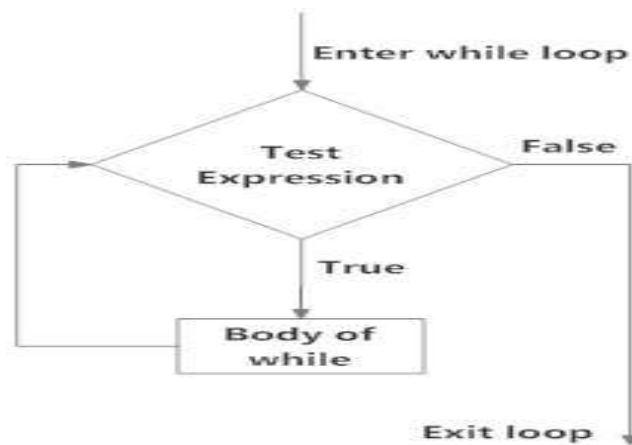
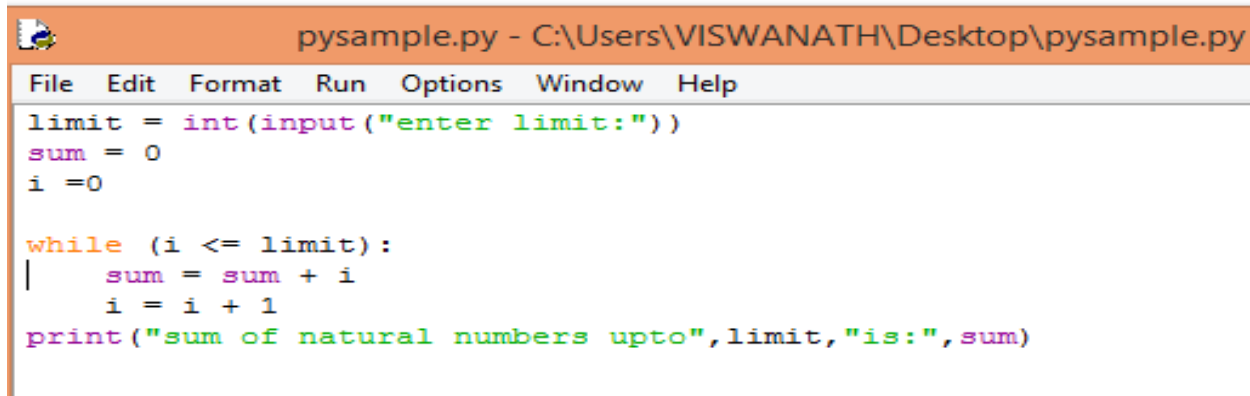


Fig: operation of while loop

- The while loop runs as long as the expression (condition) evaluates to True and execute the program block.
- The condition is checked every time at the beginning of the loop and the first time when the expression evaluates to False, the loop stops without executing any of the remaining statement(s).
- while loop tests its condition before the body of the loop (block of program statements) is executed. If the initial test returns false, the body is not at all executed .

The following example is to find the sum of n natural numbers or $1 + 2 + 3 + \dots + n$.

```
===== RESTART: C:\Users\VISWANATH\Desktop\pysample.py =
enter limit:10
sum of natural numbers upto 10 is: 55
>>>
```



```
pysample.py - C:\Users\VISWANATH\Desktop\pysample.py
File Edit Format Run Options Window Help
limit = int(input("enter limit:"))
sum = 0
i = 0

while (i <= limit):
|     sum = sum + i
    i = i + 1
print("sum of natural numbers upto", limit, "is:", sum)
```

For example the following code never prints out anything as the condition evaluates to false.

```
x = 10;
while (x < 5):
    print(x)
    x += 1
```

The following while loop is an infinite loop, using True as the condition:

```
x = 10;
while (True):
    print(x)
    x += 1
```

Python: while and else statement

- There is a structural similarity between while and else statement. Both have a block of statement(s) which is only executed when the condition is true.
- The difference is that the block belongs to if statement executes once whereas block belongs to while statement executes repeatedly. You can attach an optional else clause with while statement.

Syntax:

while (expression) :

statement_1

statement_2

.....

else :

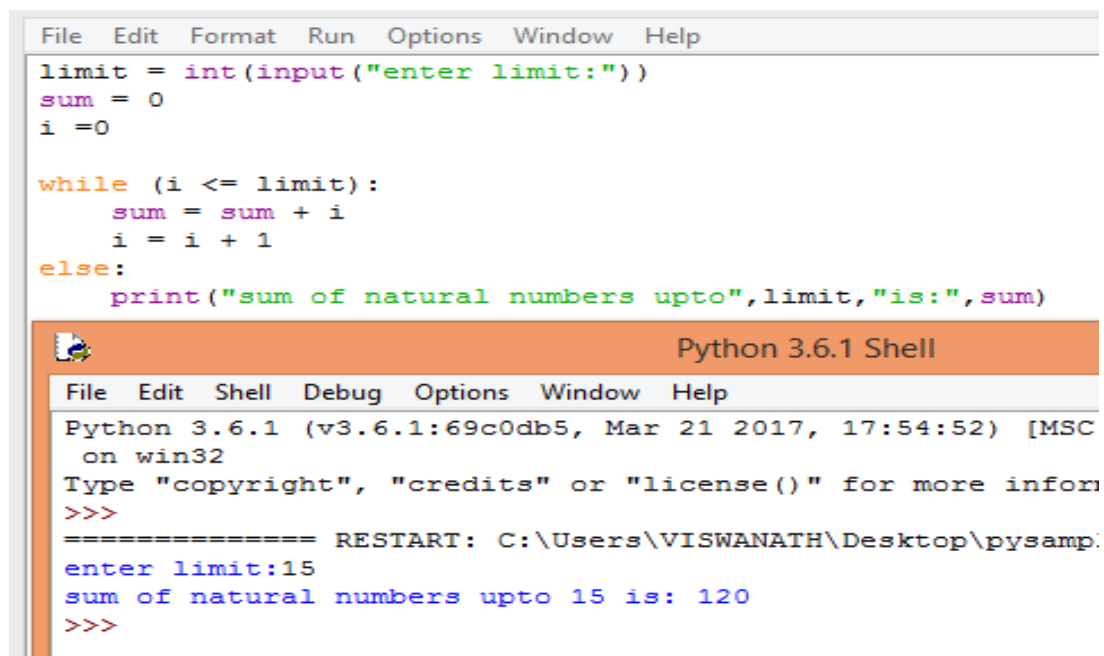
statement_3

statement_4

.....

- The while loop repeatedly tests the expression (condition) and, if it is true it executes the first block of program statements.
- The else clause is only executed when the condition is false. It will not execute if the loop breaks, or if an exception is raised.
- If a break statement executes in first program block and terminates the loop then the else clause is not executed.

In the following example, while loop calculates the sum of the integers from 0 to 9 and after completing the loop, the else statement executes.



```
File Edit Format Run Options Window Help
limit = int(input("enter limit:"))
sum = 0
i = 0

while (i <= limit):
    sum = sum + i
    i = i + 1
else:
    print("sum of natural numbers upto",limit,"is:",sum)
```

```
Python 3.6.1 Shell
File Edit Shell Debug Options Window Help
Python 3.6.1 (v3.6.1:69c0db5, Mar 21 2017, 17:54:52) [MSC
on win32
Type "copyright", "credits" or "license()" for more inform
>>>
===== RESTART: C:\Users\VISWANATH\Desktop\pysamp.
enter limit:15
sum of natural numbers upto 15 is: 120
>>>
```

3.1.3.6 BREAK

- The break statement can alter the flow of execution of normal loop. It is mainly used to exit a for or a while loop.
- The purpose of this statement is to end the execution of the loop (for or while) immediately and the program control goes to the statement after the last statement of the loop.
- If there is an optional else statement in while or for loop it skips the optional clause also. When break statement is encountered then the control of the loop is
- exited.

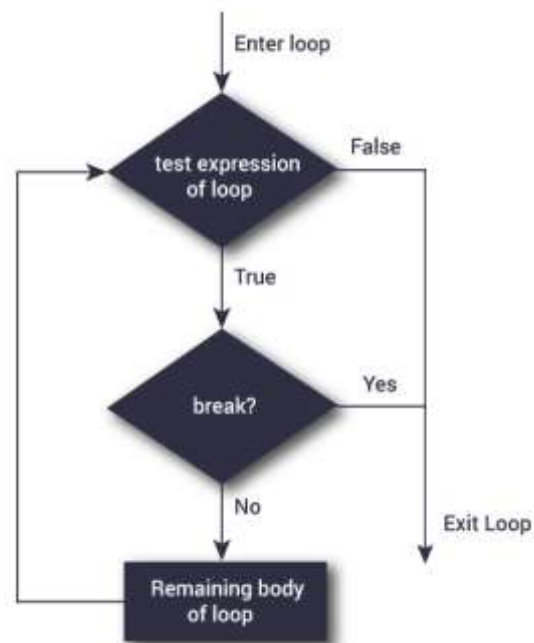
Syntax: break

```

for var in sequence:
    # codes inside for loop
    if condition:
        break
    # codes inside for loop
# codes outside for loop

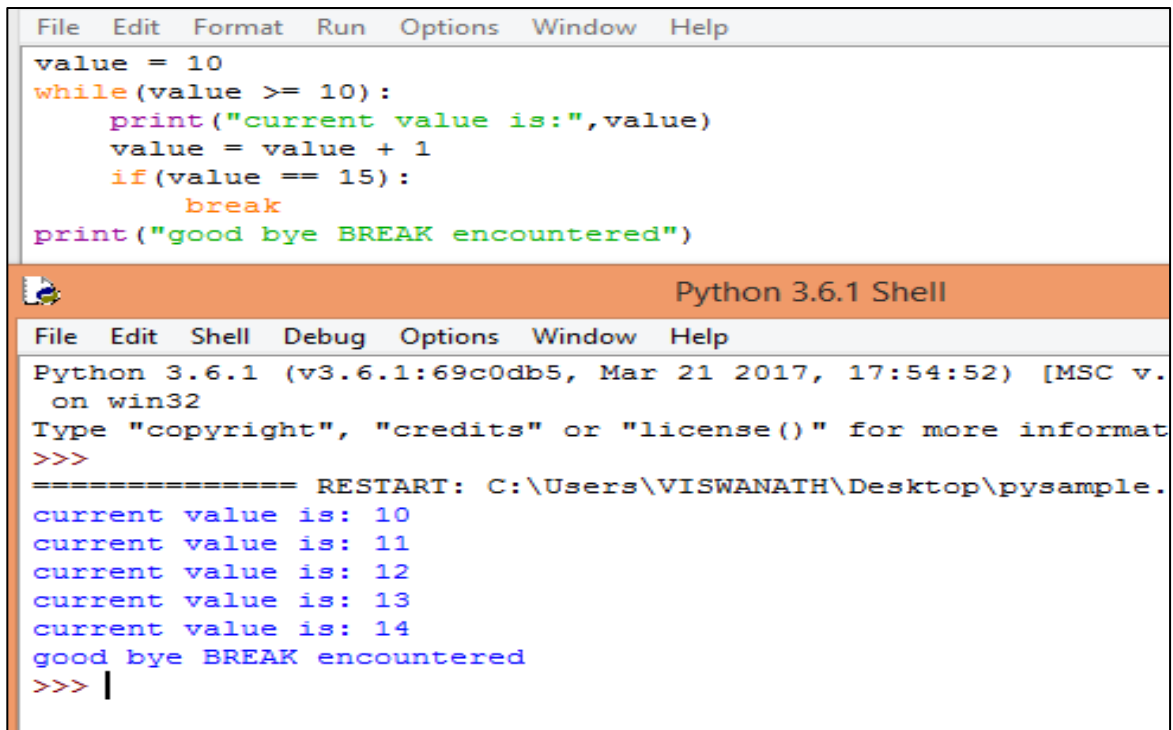
while test expression:
    # codes inside while loop
    if condition:
        break
    # codes inside while loop
# codes outside while loop
  
```

Flowchart



Example: break in while loop

In the following example while loop breaks when the count value is 15.



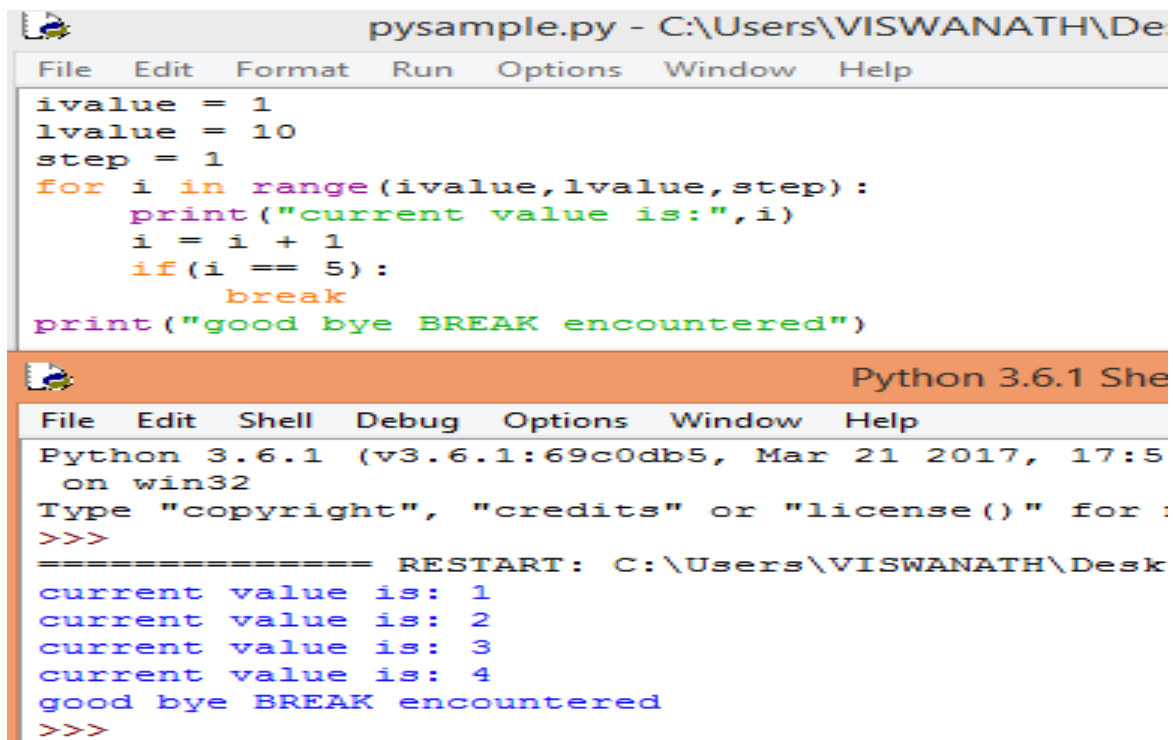
```
File Edit Format Run Options Window Help
value = 10
while(value >= 10):
    print("current value is:",value)
    value = value + 1
    if(value == 15):
        break
print("good bye BREAK encountered")
```

Python 3.6.1 Shell

```
File Edit Shell Debug Options Window Help
Python 3.6.1 (v3.6.1:69c0db5, Mar 21 2017, 17:54:52) [MSC v.
on win32
Type "copyright", "credits" or "license()" for more informat
>>>
===== RESTART: C:\Users\VISWANATH\Desktop\pysample.
current value is: 10
current value is: 11
current value is: 12
current value is: 13
current value is: 14
good bye BREAK encountered
>>> |
```

Example: break in for loop

In the following example for loop breaks when the count value is 5.



```
pysample.py - C:\Users\VISWANATH\De
File Edit Format Run Options Window Help
ivalue = 1
lvalue = 10
step = 1
for i in range(ivalue,lvalue,step):
    print("current value is:",i)
    i = i + 1
    if(i == 5):
        break
print("good bye BREAK encountered")
```

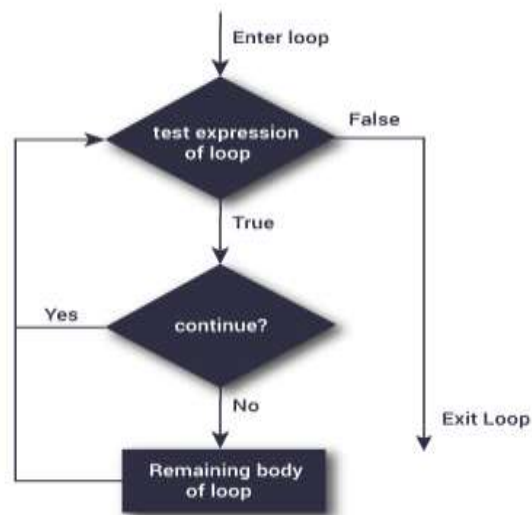
Python 3.6.1 She

```
File Edit Shell Debug Options Window Help
Python 3.6.1 (v3.6.1:69c0db5, Mar 21 2017, 17:5
on win32
Type "copyright", "credits" or "license()" for :
>>>
===== RESTART: C:\Users\VISWANATH\Desk
current value is: 1
current value is: 2
current value is: 3
current value is: 4
good bye BREAK encountered
>>>
```

3.2.2 CONTINUE

The continue statement is used in a while or for loop to take the control to the top of the loop without executing the iteration inside the loop. Usually continue does not terminate the loop execution

Syntax: continue



Example:

The screenshot shows two windows from a Python IDE. The top window, titled 'pysample.py - C:\Users\VISWANATHA', contains the following code:

```
for x in range(6):  
    if (x == 3 or x == 6):  
        continue  
    print(x)
```

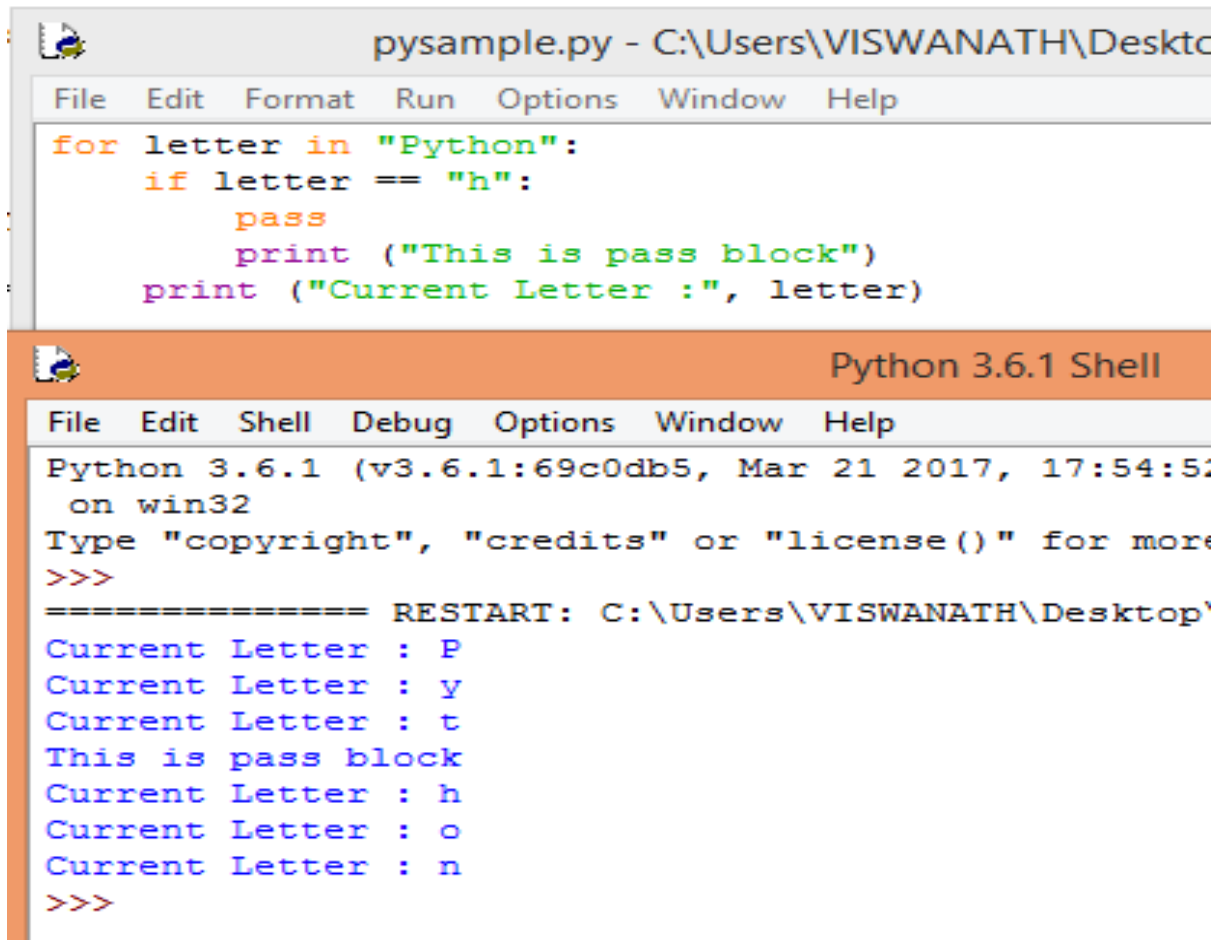
The bottom window, titled 'Python 3.6.1 (v3.6.1:69c0db5, Mar 21 2017, on win32)', shows the output of the script. It displays the numbers 0, 1, 2, 4, and 5, each on a new line, indicating that the loop skipped the values 3 and 6 due to the 'continue' statement. The prompt '>>>' is visible at the bottom of the window.

In the above example, the for loop prints all the numbers from 0 to 6 except 3 and 6 as the continue statement returns the control of the loop to the top.

3.2.3 PASS

- In Python programming, pass is a null statement.
- The difference between a comment and pass statement in Python is that, the interpreter ignores a comment entirely whereas pass is not ignored. However, nothing happens when pass is executed. It results into no operation (NOP).
- Suppose we have a loop or a function that is not implemented yet, but we want to implement it in the future. It cannot have an empty body. The interpreter would complain. So, we use the pass statement to construct a body that does nothing.

Syntax: pass



The image shows two windows from a Python IDE. The top window, titled 'pysample.py - C:\Users\VISWANATH\Desktop', contains the following Python code:

```
for letter in "Python":  
    if letter == "h":  
        pass  
        print ("This is pass block")  
    print ("Current Letter :", letter)
```

The bottom window, titled 'Python 3.6.1 Shell', shows the output of running this code. It displays the letters of 'Python' in order, followed by the message 'This is pass block' when the letter 'h' is encountered.

```
Python 3.6.1 (v3.6.1:69c0db5, Mar 21 2017, 17:54:52)  
on win32  
Type "copyright", "credits" or "license()" for more  
>>>  
===== RESTART: C:\Users\VISWANATH\Desktop\>  
Current Letter : P  
Current Letter : y  
Current Letter : t  
This is pass block  
Current Letter : h  
Current Letter : o  
Current Letter : n  
>>>
```

ILLUSTRATIVE PROGRAMS:**1. Example program for Swapping of two numbers:**

```

File Edit Format Run Options Window Help
#swapping values of two numbers
#method1
number1 = eval(input("Enter the first number: "))
number2 = eval(input("Enter the second number: "))

temp=number1
number1=number2
number2=temp

print("After swapping value of number1 is:",number1," and value of number2 is:",number2)

#method2
number1 = eval(input("Enter the first number: "))
number2 = eval(input("Enter the second number: "))
number1,number2=number2,number1
print("After swapping value of number1 is:",number1," and value of number2 is:",number2)

```

```

Python 3.6.1 Shell
File Edit Shell Debug Options Window Help
Python 3.6.1 (v3.6.1:69c0db5, Mar 21 2017, 17:54:52) [MSC v.1900 32 bit] on win32
Type "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/VISWANATH/Desktop/pysample.py =====
Enter the first number: 5
Enter the second number: 4
After swapping value of number1 is: 4 and value of number2 is: 5
Enter the first number: 1
Enter the second number: 6
After swapping value of number1 is: 6 and value of number2 is: 1
>>> |

```

2. Python program to check if the input year is a leap year or not

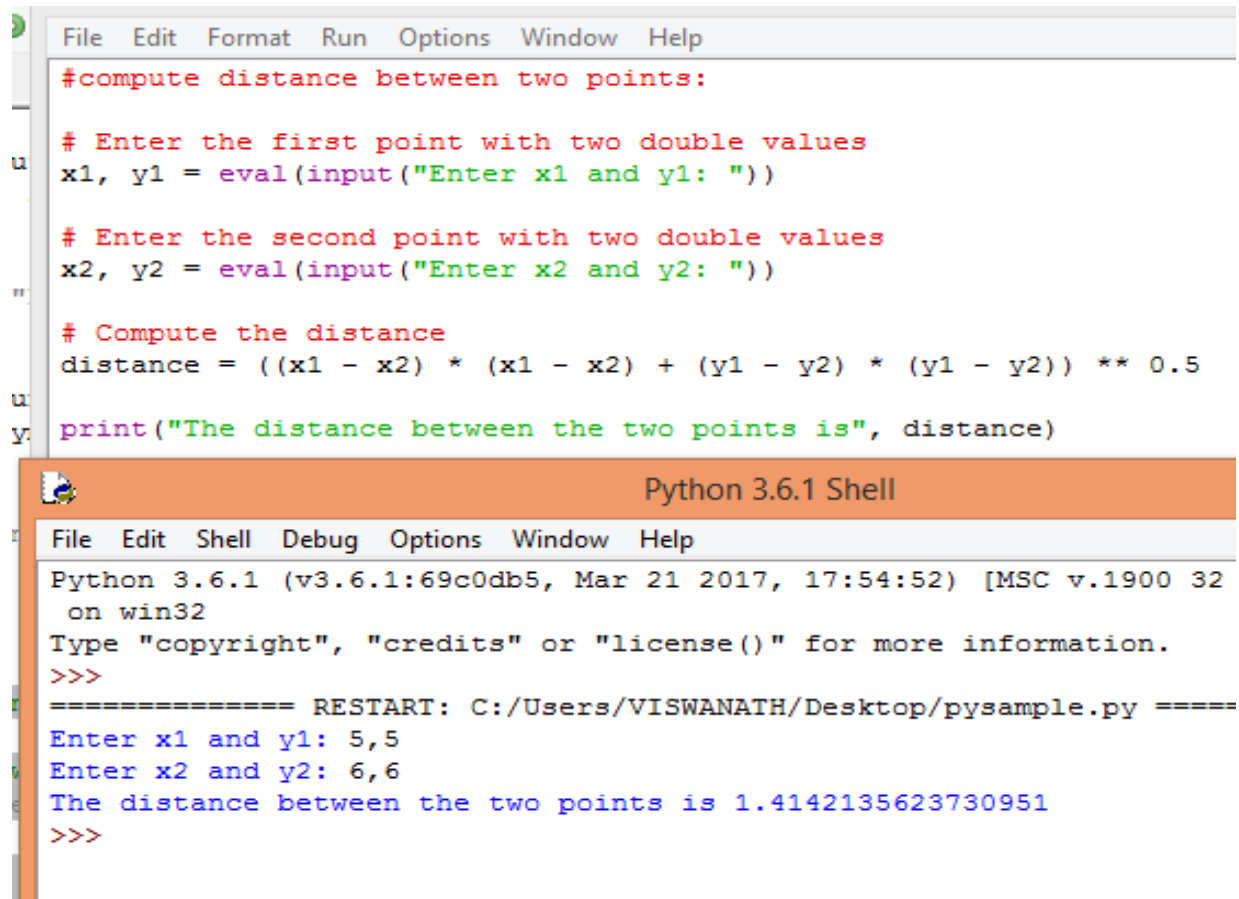
```

File Edit Format Run Options Window Help
year = eval(input("Enter a year: "))
if((year%4 == 0 and (year%100 != 0 or year%400 == 0)):
    print(year, "is a leap year")
else:
    print(year, "not a leapyear")

Python 3.6.1 Shell
File Edit Shell Debug Options Window Help
Python 3.6.1 (v3.6.1:69c0db5, Mar 21 2017, 17:54:52) [MSC v.1900 32 bit] on win32
Type "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/VISWANATH/Desktop/pysample.py =====
Enter a year: 2012
2012 is a leap year
>>>
===== RESTART: C:/Users/VISWANATH/Desktop/pysample.py =====
Enter a year: 2013
2013 not a leapyear
>>>

```

3. Python program to find distance between two points:



```
File Edit Format Run Options Window Help
#compute distance between two points:

# Enter the first point with two double values
x1, y1 = eval(input("Enter x1 and y1: "))

# Enter the second point with two double values
x2, y2 = eval(input("Enter x2 and y2: "))

# Compute the distance
distance = ((x1 - x2) * (x1 - x2) + (y1 - y2) * (y1 - y2)) ** 0.5

print("The distance between the two points is", distance)
```

```
Python 3.6.1 Shell
File Edit Shell Debug Options Window Help
Python 3.6.1 (v3.6.1:69c0db5, Mar 21 2017, 17:54:52) [MSC v.1900 32
on win32
Type "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/VISWANATH/Desktop/pysample.py =====
Enter x1 and y1: 5,5
Enter x2 and y2: 6,6
The distance between the two points is 1.4142135623730951
>>>
```

4. ITERATOR AND GENERATOR

4.1 ITERATOR

Iterators are methods that iterate collections like lists, tuples, etc. Using an iterator method, we can loop through an object and return its elements. Technically, a Python iterator object must implement two special methods, `__iter__()` and `__next__()`, collectively called the iterator protocol.

4.1.1 Iterating Through an Iterator

In Python, we can use the `next()` function to return the next item in the sequence.

Example,

```
# define a list
my_list = [4, 7, 0]
# create an iterator from the list
iterator = iter(my_list)
# get the first element of the iterator
print(next(iterator)) # prints 4
# get the second element of the iterator
print(next(iterator)) # prints 7
# get the third element of the iterator
print(next(iterator)) # prints 0
```

Output

```
4
7
0
```

Here, first we created an iterator from the list using the iter() method. And then used the next() function to retrieve the elements of the iterator in sequential order.

4.1.2 Using for Loop

A more elegant way of automatically iterating is by using the for loop. For example,

```
# define a list
my_list = [4, 7, 0]
for element in my_list:
    print(element)
```

Output

```
4
7
0
```

Working of for loop for Iterators

The for loop in Python is used to iterate over a sequence of elements, such as a list, tuple, or string. When we use the for loop with an iterator, the loop will automatically iterate over the elements of the iterator until it is exhausted.

Example:

```
# create a list of integers
my_list = [1, 2, 3, 4, 5]
# create an iterator from the list
iterator = iter(my_list)
# iterate through the elements of the iterator
for element in iterator:
    # Print each element
    print(element)
```

Output

```
1
2
3
4
5
```

In this example, the ‘for’ loop iterates over the elements of the iterator object. On each iteration, the loop assigns the value of the next element to the variable element, and then executes the indented code block. This process continues until the iterator is exhausted, at which point the for loop terminates.

4.2 GENERATOR

In Python, a generator is a function that returns an iterator that produces a sequence of values when iterated over.

4.2.1 Create Python Generator

In Python, similar to defining a normal function, we can define a generator function using the def keyword, but instead of the return statement we use the **yield** statement.

```
def generator_name(arg):
```

```
    # statements
```

```
    yield something
```

Here, the yield keyword is used to produce a value from the generator.

When the generator function is called, it does not execute the function body immediately. Instead, it returns a generator object that can be iterated over to produce the values.

Example: Python Generator

Here's an example of a generator function that produces a sequence of numbers,

```
def my_generator(n):
```

```
    # initialize counter
```

```
    value = 0
```

```
    # loop until counter is less than n
```

```
    while value < n:
```

```
        # produce the current value of the counter
```

```
        yield value
```

```
        # increment the counter
```

```
        value += 1
```

```
# iterate over the generator object produced by my_generator
```

```
for value in my_generator(3):
```

```
    # print each value produced by generator
```

```
    print(value)
```

Output

```
0
```

```
1
```

```
2
```

- In the above example, the `my_generator()` generator function takes an integer `n` as an argument and produces a sequence of numbers from 0 to `n-1`.
- The `yield` keyword is used to produce a value from the generator and pause the generator function's execution until the next value is requested.
- The `for` loop iterates over the generator object produced by `my_generator()`, and the `print` statement prints each value produced by the generator.

We can also create a generator object from the **generator function** by calling the function like we would any other function as,

```
generator = my_range(3)
print(next(generator)) # 0
print(next(generator)) # 1
print(next(generator)) # 2
```

4.2.2 Python Generator Expression

In Python, a generator expression is a concise way to create a generator object. It is similar to a list comprehension, but instead of creating a list, it creates a generator object that can be iterated over to produce the values in the generator.

Generator Expression Syntax

A generator expression has the following syntax,:- **(expression for item in iterable)**

Here, `expression` is a value that will be returned for each item in the `iterable`.

The generator expression creates a generator object that produces the values of `expression` for each item in the `iterable`, one at a time, when iterated over.

Example : Python Generator Expression

```
# create the generator object
squares_generator = (i * i for i in range(5))
# iterate over the generator and print the values
for i in squares_generator:
    print(i)
```

Output

0
1
4
9
16

Here, we have created the generator object that will produce the squares of the numbers 0 through 4 when iterated over. And then, to iterate over the generator and get the values, we have used the for loop.

4.3 ITERATOR Vs GENERATOR

Iterator	Generator
Class is used to implement an iterator	Function is used to implement a generator.
Local Variables aren't used here.	All the local variables before the yield function are stored.
Iterators are used mostly to iterate or convert other objects to an iterator using iter() function.	Generators are mostly used in loops to generate an iterator by returning all the values in the loop without affecting the iteration of the loop
Iterator uses iter() and next() functions	Generator uses yield keyword
Every iterator is not a generator	Every generator is an iterator